

VeriSSO: A Privacy-Preserving OIDC-Compatible Single Sign-On Protocol Using Verifiable Credentials

Ifteher Alom, Sudip Bhujel, Yang Xiao

Department of Computer Science, University of Kentucky, Lexington, KY, USA
{ifteheralom, sudipbhujel, xiaoy}@uky.edu

Abstract—Single Sign-On (SSO) is a popular authentication mechanism enabling a user to access different online services (called Relying Parties, or RPs) with a single login credential obtained from the Identity Provider (IdP). Despite its convenience, SSO schemes represented by the OIDC standard faces significant privacy concerns—the IdP can track users across different RPs; colluding RPs may share data to find linkage of user access. Recent anonymous credential-based SSO solutions provide a promising direction to enhancing user privacy and mitigating IdP single-point failure; however, they fail to support RP authentication, an important security property of the incumbent SSO workflow, and require RPs to perform non-trivial cryptographic verification.

This paper introduces VeriSSO, a novel privacy-preserving SSO protocol based on verifiable credentials (VC) that supports RP authentication and is fully compatible with the incumbent SSO workflow. The key intuition is to employ a committee of independent authentication servers (i) to bind RP authentication to VC-based user verification and (ii) to issue identity tokens in a threshold manner, which crucially ensures RP authentication and user unlinkability without IdP involvement or reliance on a trusted central party. Our scheme allows RPs to continue using their existing signature-based identity token verification routine and supports lawful de-anonymization, providing user accountability for misbehavior. Our experiment shows the feasibility and efficiency and VeriSSO, with one SSO workflow completed within 30 milliseconds.

Index Terms—Single Sign-On, OpenID Connect, Privacy, Authentication, Verifiable Credentials.

I. INTRODUCTION

Single Sign-On (SSO) is a widely used authentication method that allows users to access various web services with a single login credential. In a typical SSO workflow, a user attempts to log into the service portal of a Relying Party (RP) by authenticating with an Identity Provider (IdP) with whom the user has a preexisting registration. Upon authenticating the user, the IdP generates an *identity token* specifying certain user attributes (e.g., name, email, age) required by the RP. The IdP signs the identity token with its private key, attesting to the user's identity to the RP. Currently, OpenID Connect (OIDC) [1] is the most commonly used SSO standard, whose identity token takes the form of a JSON Web Token (JWT) derived from OAuth 2.0's access token [2].

Despite SSO's convenience and wide adoption, particularly the OIDC standard, it faces increasing privacy and security challenges. First, an IdP's central role in the authentication

flow poses a significant privacy risk to users. The user has to inform their IdP about which RP will be accessed, enabling the IdP to track the user's accesses to different RPs at all times, contributing to the *IdP linkability* risk. By analyzing the nature of each RP website, a curious IdP can infer sensitive activities of the user, leading to privacy violation [3]. The IdP also represents a *single point of failure*, since it is fully responsible for managing and verifying user login credentials. The IdP must always be online to authenticate users and issue identity tokens; any IdP service downtime would hinder its users from timely access to RP services [4], [5]. Second, SSO users face privacy threats from curious and potentially colluding RPs [6], [7]. The identity tokens generated by an IdP for a user often contain long-term user attributes, such as email, zip code, and affiliation, contributing to the *RP linkability* risk. These attributes, when exposed to a group of colluding RPs in separate SSO sessions, allow the RPs to identify and track the user [8], [9].

Recently, anonymous credentials (AC) have emerged as a promising solution for enabling privacy-preserving SSO [10], [11], [12], [13], [14], [15], particularly for addressing the user linkability challenge, and mitigating IdP single-point failures. AC is a type of attribute-based cryptographic credential that enables a user to prove certain identity attributes to a verifier without revealing unnecessary information, which has evolved independently in the past [16], [17], [18], [19]. When AC is used for SSO, a user typically obtains a digitally signed credential containing a list of identity attributes from the IdP. During a login session with an RP, the user selectively discloses the credential's attributes as required by the RP; no unnecessary attributes are disclosed. A zero-knowledge proof (ZKP) is often used to attest to the authenticity and integrity of the disclosed attributes (as they are inherited from a valid credential) without revealing linkage to the original credential or other showings of the same credential.

A. Challenges

While AC-based solutions offer appealing privacy and robustness benefits, they often come at the price of the essential security properties and computational efficiency. We describe the two main challenges.

Challenge-1: RP authentication vs. unlinkability. Existing AC-based SSO schemes lack support for RP authentication, which is an important incumbent function for protecting

users from illegal content and phishing scams by unauthorized RPs. In OIDC [1], RP authentication is mandatory for the *Authorization Code Flow* (ACF), a major type of SSO flow where the RP must authenticate to the IdP before receiving the identity token about the user. In particular, it requires a user to identify the RP to the IdP. If RP authentication is added to AC-based SSO schemes trivially, it would reintroduce the user linkage problem since the IdP could easily track the user’s access requests to different RPs, forfeiting the unlinkability property once achieved through anonymous authentication. For this reason, most existing AC-based SSO schemes [10], [11], [13], [15] are only designed to support the *Implicit Flow* (IF), which does not require RP authentication nor any interaction between the RP and the IdP. However, IF has been deprecated by OIDC exactly due to the lack of RP authentication among other security risks, making ACF the dominant flow. A viable solution should support RP authentication without creating new risks of user linkage.

Challenge-2: Anonymity vs. ACF compatibility. AC-based SSO schemes require an RP to verify a user anonymously, which commonly involves verifying a zero-knowledge proof (ZKP) of certain identity attributes in a credential showing. Such a verification task is significantly more costly than the existing practice in OIDC’s ACF workflow, where an RP only needs to verify the standard public-key signature in an identity token. This computational overhead not only leads to performance bottlenecks when the RP has to handle millions of user requests, but also, to some extent, defeats the purpose of “single sign-on” since each user-RP interaction now constitutes a standalone authentication process [20]. A practical solution would allow RPs to continue using their existing verification routines (i.e., verifying signatures in identity tokens in ACF) for efficiency.

Recent privacy-preserving SSO proposals positioned as OIDC extensions have partially addressed the above challenges. AIF [21] extends the Implicit Flow with AC-based RP authentication to achieve RP hiding—the IdP does not know which exact RP is accessed by the user. This approach, however, does not fully translate to user unlinkability and requires RPs to get involved in AC management and presentation, which brings non-trivial overhead. MISO [20] leverages trusted execution environment (TEE) to establish a trusted identity mixing service to perform RP authentication for privacy-sensitive users. It nonetheless requires full trust in the TEE hardware and does not support selective disclosure of user attributes.

B. Contributions

We introduce VeriSSO, a privacy-preserving SSO protocol that supports RP authentication and is fully compatible with the incumbent OIDC ACF workflow. VeriSSO is based on Verifiable Credentials (VC), a general formulation for secure credentials enveloping AC¹ and consists of two phases. In the registration phase, each user receives from their IdP a VC containing a list of long-term attributes. In the authentication

phase, the VC can be used to compute verifiable presentations (VPs) that selectively disclose the user’s attributes along with ZKPs attesting to the VP’s validity. The key novelty of VeriSSO is to employ an independent committee of *authentication servers* (ASs) to provide RP authentication, VP-based user authentication, as well as identity token generation in a threshold manner.

Binding RP Authentication to Anonymous User Authentication. To perform RP authentication without creating user linkage, we require a user to present the RP authenticator, usually a message signed by RP, along with a VP disclosing the minimally required user attributes to each AS (instead of being disclosed to the IdP as in the incumbent ACF). This design essentially allows each AS to bind VP-based anonymous user authentication with RP authentication, which will take place later when the RP requests the identity token through the RP-AS back channel. Since the VP cannot be linked to a specific user due to properties of ZKP, the ASs cannot learn the user’s real identity or RP accessing patterns, thus giving users unlinkability against colluding RPs. The IdP is not involved in any stage of the user authentication flow and thus does not pose a linkage risk. Our VP design also supports *lawful de-anonymization* to hold a user accountable if anonymity is misused. The user identifier is encrypted with ZKP-attested correctness in the VP and can be decrypted by a threshold majority of ASs.

Threshold-based Generation of Identity Token with ACF Compatibility. After each authentication server verifies the user and RP, VeriSSO requires the RP to obtain the user identity token that carries the disclosed attributes in a threshold manner. The user first obtains an access token τ_{ac} from each authentication server and forwards it to the RP. The RP can then use the access token to retrieve the identity token, τ_{id} , and a *share* of its threshold signature, from each authentication server, which contains a share of the public key signature generated through a multiparty computation (MPC) protocol between the authentication servers. This RP-AS communication pattern resembles the back-channel identity token retrieval between the RP and IdP in ACF. For the RP, τ_{id} can be reconstructed from a threshold fraction of the identity token shares. Throughout the process, there is no single point of failure for the generation of τ_{id} , and its correctness is guaranteed as long as the threshold fraction of AS behaves correctly. In the end, the RP verifies the signature in the reconstructed τ_{id} just like in ACF.

In summary, we make the following contributions with VeriSSO:

- At the functional level, VeriSSO is the first credential-based, privacy-preserving SSO solution that simultaneously supports RP authentication, prevents user identity linkage, and enables lawful de-anonymization. It also allows RPs to continue their existing verification routine in OIDC, i.e., verifying the user authentication success by checking the digital signature in an identity token.
- On the design level, VeriSSO binds RP authentication to VC-based user authentication and integrates threshold issuance of identity tokens into the ACF workflow. This

¹For a consistent narrative, we adopt the W3C’s VC framework [22] to describe AC functionalities, though AC evolved independently before VC.

crucially provides resistance to single-point failures while ensuring RP authentication and user unlinkability, without reliance on trusted hardware.

- We implemented and open-sourced a VeriSSO prototype. Our cloud-based experiments show that a complete SSO authentication workflow finishes within 30 milliseconds, demonstrating a reasonable time cost for practical environments.

Paper Organization. The rest of the paper is organized as follows. In §II we present the background of our scheme. In §III we describe our system model and an overview. In §IV, we introduce the cryptographic building blocks of our system. We present the detailed design of VeriSSO in §V and analyses in §VI. In §VII we describe our implementation and evaluation results. We discuss and compare related work in §VIII, followed by conclusions in §IX.

II. BACKGROUND

A. Sign Sign-On

In a typical setting, as shown in Fig. 1, a user initiates an SSO authentication session usually after selecting the “login with SSO” option when attempting to access the RP’s service portal. In response, the RP produces an SSO request for the session containing its membership/registration identity rid with the IdP, authentication-required attributes, service endpoints, and PKI information, and then redirects the user to their IdP for the actual authentication, which can involve normal login methods such as username-password or biometrics. Once authenticated, the IdP produces a signed **identity token** (τ_{id}) and delivers it to the RP. The RP extracts the user identity attributes, e.g., email and institution, from the identity token and verifies their legitimacy with the IdP’s signature. As a result, the RP does not need to authenticate the user independently, and the user does not need to keep separate login credentials for different RPs. SSO also enables a user to authorize resource access from across different providers. For example, a user may request an RP to download their profile data, images, credit score, and other information from remote servers. The user authorizes the host server using an **access token** (τ_{ac}) that allows it to fetch the required data. In the actual SSO deployment, represented by OIDC [1], there exist two main types of workflows.

The **Implicit Flow** (IF) was developed around 2010 when mobile app platforms were new, and single-page apps were emerging. Browsers at that time did not support cross-domain POST requests. The access token is returned in the redirect URL, which can be intercepted if the redirect is compromised. Examples of vulnerabilities include captive Wi-Fi portals intercepting redirects or browser extensions that can access the address bar and log URLs. Extensions can monitor and capture access tokens from the URL. Browser sync features (like Chrome syncing) can propagate access tokens across multiple devices, increasing the risk of theft.

The **Authorization Code Flow** (ACF) involves a two-step process (depicted in Figure 1) where an access token τ_{ac} is received and then exchanged for an access token, providing an extra layer of security. By not returning the access token

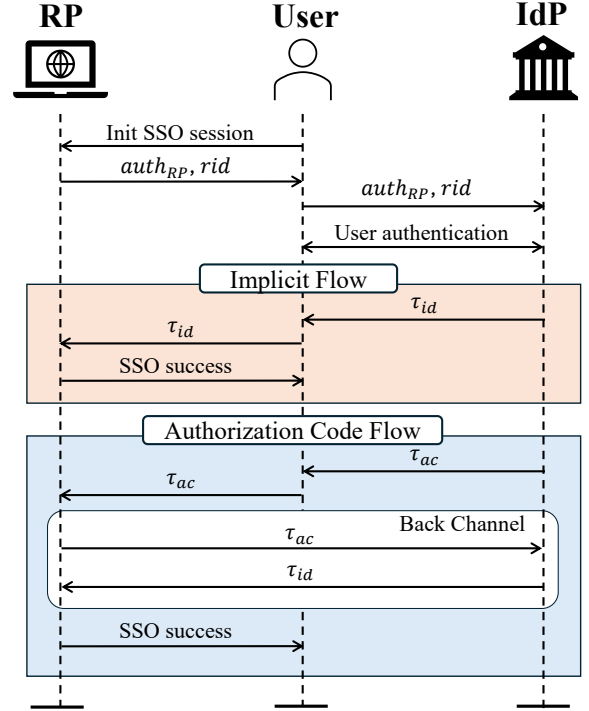


Fig. 1: OIDC authentication workflows [1].

directly in the redirect, the ACF reduces the risk of interception during the initial request. Modern browsers now support cross-origin resource sharing (CORS) policies, enabling secure cross-domain POST requests needed for the authorization code flow. Moreover, existing ACF implementations take advantage of Proof Key for Code Exchange (PKCE) [23] to enhance security, especially for public clients (e.g., mobile apps), by using a dynamically generated cryptographically random secret during the authorization process, making it harder for attackers to use intercepted codes. Most importantly, RP authentication is not required for IF but is mandatory for ACF [21]. As a result, IF is no longer recommended, with ACF being the dominant SSO standard [24], [25], [26].

B. Verifiable Credential

A Verifiable Credential (VC) is a digitally signed document containing user attributes that support cryptographic verification and tamper-evident integrity [27], [22]. Users store VCs locally and create Verifiable Presentations (VPs) to share verified attributes with service providers or verifiers, who confirm authenticity before granting access. Anonymous Credential (AC), though independently evolved in the past [16], [17], [18], [19], is considered as a special VC under the W3C’s VC framework [22] to enhance user privacy by enabling selective attribute disclosure through zero-knowledge proofs (ZKP), allowing sensitive information such as identifiers or age to remain hidden [28]. AC ensures the anonymity and unlinkability of multiple VPs derived from the same credential, preventing signature-based user tracking. Typically, AC schemes are implemented using multi-message short signature

TABLE I: List of Notations.

Notation	Description
$auth_{RP}$	Authentication request
rid	RP's identity with IdP/TAL
sch_{VP}	Schema requirements
τ_{ac}	Access token
τ_{id}	Identity token
N	Number of ASs in our system
t	Threshold parameter for ASs
$cred$	Verifiable Credential (VC)
$pres$	Verifiable Presentation (VP)
$\vec{m}$	Attribute set in VC
u	User-side VC identifier
γ	IdP-assigned VC identifier
σ	BBS signature
σ_T	Threshold BBS+ signature
$\tilde{u}$	RP-specific pseudonym
ϕ	ZKP of VC identifiers (γ, u)

schemes like CL signatures [29], [30], [31] and BBS signatures [32], [33], coupled with efficient ZKP methods.

III. SYSTEM OVERVIEW

A. Participants

We consider three principals in a typical SSO workflow: identity provider (IdP), relying party (RP), and user:

- **IdP** is an entity that maintains a trustworthy identity database of registered users and RPs. It is capable of generating public key signatures over a registered identity attesting to its validity. The IdP also registers RPs that want to delegate their authentication services and their policies in a Trusted Anchor List (TAL). Common IdPs include web service platforms (e.g., Google and Meta), universities, and specialized identity management services (e.g., Okta).
- **RP** offers online services/resources to users. RPs register their service terms with IdPs, stating their own digital identities, resource URLs, and user attributes policy (i.e., what identity attributes are required for providing service to a user).
- **User** is a customer who wants to access online services/resources from the RP via the SSO option. A user has a pre-existing registration with the IdP, specifying their long-term digital identity and a full list of attributes.

In addition to the principals, VeriSSO introduces two new entity types to fulfill our design goals: authentication server (AS) and verifiable data registry (VDR).

- An **AS** is capable of verifying the validity of users and RPs. The incumbent SSO standards, such as OIDC, do not require a standalone AS since the IdP verifies users and RPs all by itself. Nonetheless, in commercial space, standalone authentication services such as Auth0 [34] and OneLogin [35] begin to gain popularity due to their benefits in alleviating IdP verification workload. Building on this intuition, VeriSSO employs a committee of N

independent ASs that operate in a (t, N) threshold manner to enforce user/RP authentication and subsequently generate identity tokens.

- The **VDR** serves as a public bulletin board for all the participants. It may include the public key certificates of the RPs and IdPs and the list of blacklisted user credentials or users. VDR also provides the ASs with access to the IdP's TAL, where it publishes the metadata of registered RPs and their service terms.

B. Threat Model

We consider the IdP and registered RPs are *semi-honest*—they correctly follow their designated routines but may attempt to infer a user's web access history and their permanent identity. Malicious, unregistered RPs may spoof users for phishing or scam purposes. We also consider registered RPs to be curious about users' sensitive identity information. Multiple RPs may collude to discover the real user identity behind different SSO sessions. Malicious unregistered users are motivated to obtain access to a registered RP. Similarly, a registered user may also wish to access RPs without passing the SSO authentication process.

We assume that $N - t$ individual ASs may be *compromised*—they may not follow the designated routine due to either system compromise or service outage. The remaining t ASs are honest (i.e., executing the designated routine correctly) and available. We also assume at most $t - 1$ ASs are *curious* about a user's access history and may collude to do so. A curious AS may also collude with RPs (to help phish users) or collude with users (to help scam RPs for service). Note that the compromised property and the curious property are independent—some ASs can be both compromised-and-curious and some can be honest-but-curious.

C. System Goals

The fundamental objective of any SSO mechanism, including ours, is **user authentication**: only legitimate users who have registered with an IdP can successfully authenticate to a RP's service, in accordance with the pre-existing service agreement between the IdP and the RP. We further aim to achieve the following privacy, security, and efficiency objectives.

Privacy Objectives:

- **Selective (Minimal) disclosure.** In each SSO session, a user only needs to disclose minimally required identity information to the RP and ASs, for instance, the association with a certain IdP. By hiding personally identifiable information from the disclosure, users can remain *anonymous* in SSO operations.
- **IdP/RP/AS unlinkability.** An SSO access attempt should be *unlinkable* by a curious IdP, RP, or AS to any specific user. Specifically, the IdP cannot decide which RPs a particular user has accessed; an RP or an AS cannot decide which specific user is making the access attempt among multiple access attempts.
- **Collusion resistance.** The above minimal disclosure and unlinkability properties should hold against any collusion

between the IdP, RPs, and ASs. This includes *inter-RP unlinkability*, which means multiple colluding RPs cannot track the access history of a user.

Security Objectives:

- **RP authentication:** Only legitimate RPs registered with an IdP can provide service to the IdP's users.
- **Intra-RP linkability.** An RP can recognize authentication attempts originating from the same user registered with the same IdP. This allows the RP to mitigate Sybil attacks and conserve server resources [11].
- **User accountability (lawful de-anonymization).** Users should be traceable and de-anonymized under the agreement of a majority of the ASs, in case an anonymized user should be held responsible for misbehavior.
- **No single point failure:** The compromise of individual IdP/AS cannot affect the integrity and availability of the SSO process in the user authentication phase. The scheme also does not have to use trusted hardware.

Efficiency Objectives:

- **ACF compatibility for RP.** RPs should be allowed to continue their incumbent SSO routines as in OIDC's ACF workflow, i.e., verifying the success of user authentication by checking the digital signature of an identity token retrieved from an authentication service. Particularly, RPs do not need to perform expensive zero-knowledge proof verification as in existing AC-based SSO solutions [10], [11], [13].

D. VeriSSO Overview

VeriSSO is based on two core designs: (i) by binding RP verification to VC-based privacy-preserving user verification, an AS can authenticate both the RP for its legal identity and the user for the validity of its credential, without learning the identity of the user; (ii) by having the ASs issue an identity token using a lightweight threshold signature scheme, the RP can conveniently reconstruct the identity token and verify it using regular public-key cryptography. Both designs also contribute to the resistance against IdP or AS single-point failures. Figure 2 provides an overview of VeriSSO's two phases, showing the communication pattern and key functions.

In the **setup phase**, RPs and users set up prerequisites with the IdP crucial for later authentication functions. RP registration is similar to the OIDC counterpart [36] in terms of metadata requirement and RP identifier (rid) generation. The difference is that instead of having the IdP store RP information locally only, VeriSSO's version requires the IdP to push the RP's identity and its associated terms/requirements as a TAL entry to the VDR, which is accessible by all ASs. RP's metadata is usually in a specified format (like XML) in existing IdP's databases [37]. It contains several pieces of information such as entity descriptor, service endpoints, public key certificates of IdP and RP, the expiration time of metadata, etc. In our system, storing RP metadata in the VDR allows the IdP to delegate the RP authentication task to the ASs, which we describe in the next section.

The user setup features the user obtaining a VC from the IdP, consisting of several attributes of the user. The attributes

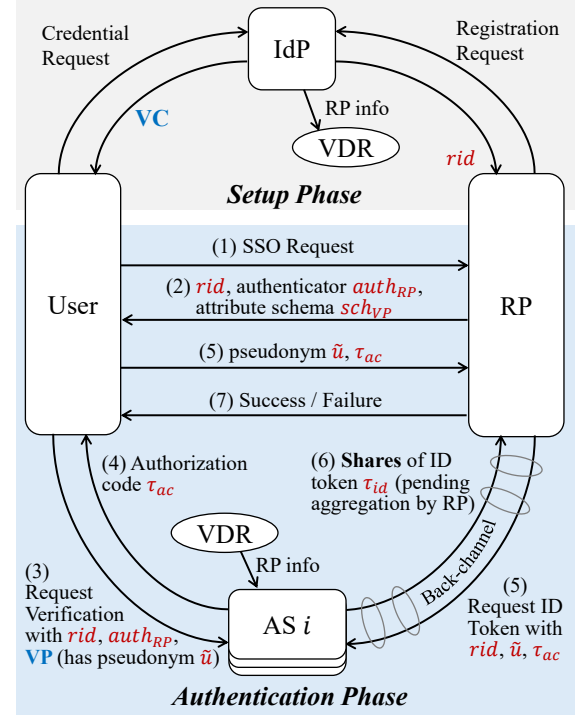


Fig. 2: VeriSSO overview.

are proposed by the user and confirmed against IdP's internal records, such as student databases, and/or physical identities such as a driver's license or passport. In addition, the user attaches a random secret u in blinded form so that the IdP does not learn the complete list of attributes, thus providing the user with an additional layer of control against collusion. The user is equipped with a digital wallet application (e.g., as a browser extension) capable of verifying and storing VCs issued by the IdP. The wallet is bound to the VC using the secret u , and the user can generate VPs based on a VC according to the user's interaction with an RP. The IdP keeps a log of the issued VC, its signature, assigned permanent VC identifier γ , and other auxiliary metadata for accountability purposes in case of VC misuse.

In the **authentication phase**, a user proves its identity to an AS committee, verifies RP, and gets access to the resources in a privacy-preserving manner. The user initiates the SSO session by accessing the login URL of the RP and receives its requirements and proof of its registration with the IdP. Then, the user relays the RP's information ($auth_{RP}$, sch_{VP}) to the AS committee for verifying the RP's claims with the TAL records on the VDR. Next, the user derives a VP, denoted $pres$, by selectively disclosing attributes from its VC according to the requirements. The disclosure also includes an RP-specific pseudonym for facilitating intra-RP linkability, which aids in Sybil mitigation by the RP later. Each AS verifies $pres$ and returns an access token τ_{ac} to the user, who waits for t responses and then forwards it to the RP. Lastly, the RP establishes a secure back-channel with the AS committee nodes (akin to the back-channel between RP and IdP servers in the incumbent ACF) to exchange the access token τ_{ac} for

the user identity token τ_{id} , which contains the disclosed user attributes in a standard JSON Web Token (JWT) format with a threshold signature. Notably, each AS node generates a *share* of τ_{id} 's signature through an efficient (t, n) -threshold signature scheme that is resilient to $(n - t)$ failures among the n AS nodes. Upon successful aggregation of τ_{id} 's signature from the shares and verification of its correctness, the RP grants access to the user.

IV. BUILDING BLOCKS

In this section, we describe the building blocks of VeriSSO. We characterize them as ideal functionalities along with realizations from existing cryptographic primitives. We will utilize the ideal functionalities to compose our protocol in the next section, abstracting away their implementation details.

A. Blindly Issued Verifiable Credential

Definition 1 (Ideal Functionality $\mathcal{F}_{\text{BIVC}}$). *The blindly issued verifiable credential ideal functionality $\mathcal{F}_{\text{BIVC}}$ provides the following interfaces:*

- $\mathcal{F}_{\text{BIVC}}.\text{prep}(\vec{m}_e, \vec{m}_s, d)$ prepares a credential issuance request with two sets of attributes: the exposed set $\vec{m}_e$ which will be seen by the issuer and the sensitive set $\vec{m}_s$ which will be blinded using a user-side blinding factor d . It returns $\vec{m}_e$ and the blinded set $\vec{m}'_s = \vec{m}_s^d$.
- $\mathcal{F}_{\text{BIVC}}.\text{issue}(\vec{m}_e, \vec{m}'_s, \gamma)$ checks for the validity of the claimed $\vec{m}_e$ and produces a signature $\hat{\sigma}$ on the attributes $\vec{m}_e$, $\vec{m}'_s$, and the credential identifier γ . It returns $\hat{\text{cred}} = (\vec{m}_e, \vec{m}'_s, \gamma, \hat{\sigma})$.
- $\mathcal{F}_{\text{BIVC}}.\text{unblind}(\hat{\text{cred}}, d)$ verifies the signature $\hat{\sigma}$ contained in $\hat{\text{cred}}$ and unblinds $\hat{\sigma}$ with the blinding factor d to obtain the signature σ . It returns the final credential $\text{cred} = (\vec{m}_e, \vec{m}_s, \gamma, \sigma)$.

To realize $\mathcal{F}_{\text{BIVC}}$, we integrate the Boneh-Boyen-Shacham (BBS) signature scheme [32] with a weak partially-blinding mechanism. BBS signature was originally proposed as a group signature scheme and has been used as a building block for VC standardization in the latest RFC draft [38]. We detail the realization of $\mathcal{F}_{\text{BIVC}}$ with BBS signature in Appendix C.

B. Selectively Disclosed Verifiable Presentation

$\mathcal{F}_{\text{SDVP}}$ essentially uses zero-knowledge proofs (ZKP) to achieve selective disclosure and its verifiability. ZKP is a general class of protocols between two parties, where the prover convinces the verifier of the validity of a given statement without revealing any information beyond the truth of the statement. By applying the Fiat-Shamir transform [39] under the Random Oracle Model [40], an interactive ZKP protocol can be transformed into a non-interactive zero-knowledge proof (NIZK) that consists of one message from the prover [41]. A particular type of NIZK, known as signature proof of knowledge (SPoK) [42], allows a prover to partially disclose messages signed by a short signature scheme (e.g., BBS) and simultaneously randomize the presentation of the original signature. We detail the realization of

$\mathcal{F}_{\text{SDVP}}$ with ZKP-augmented BBS signature in Appendix D.

Definition 2 (Ideal Functionality $\mathcal{F}_{\text{SDVP}}$). *The selectively disclosed verifiable presentation ideal functionality $\mathcal{F}_{\text{SDVP}}$ provides the following interfaces:*

- $\mathcal{F}_{\text{SDVP}}.\text{create}(\text{cred}, \vec{m}_d, \vec{m}_h, \text{info})$ creates a selectively disclosed verifiable credential *pres* for the authentication request auth_{RP} with two effects:
 - 1) It encrypts the cred's permanent identifier γ into ciphertext C_γ and generates the credential pseudonym $\tilde{u}$ from the hash of $\vec{m}_h$ and info , along with an NIZK ϕ that proves C_γ and $\tilde{u}$ are well-formed.
 - 2) It hides sensitive attributes $\vec{m}_h$ and discloses required attributes $\vec{m}_d$ from the original cred's attribute set $\vec{m}$, according to the requirements sch_{VP} , and construct a proof of knowledge of cred's signature σ (SPoK), denoted as π .

It returns the verifiable presentation $\text{pres} = (\vec{m}_d, C_\gamma, \tilde{u}, \phi, \pi)$.

- $\mathcal{F}_{\text{SDVP}}.\text{verify}(\text{pres})$ verifies the proofs ϕ and π contained in *pres*. It checks ϕ for the correctness of the obscured identifiers C_γ and $\tilde{u}$, and checks π for the validity of the disclosed attributes $\vec{m}_d$. It returns SUCCESS if both verifications pass; FAIL otherwise.

C. Threshold Identity Tokens

Definition 3 (Ideal Functionality $\mathcal{F}_{\text{TIDT}}^{(t, N)}$). *The ideal (t, N) -threshold identity token issuance functionality $\mathcal{F}_{\text{TIDT}}^{(t, N)}$ provides two interfaces:*

- $\mathcal{F}_{\text{TIDT}}^{(t, N)}.\text{issue}(\tau_{ac}, \vec{m}_d, \text{auth}_{\text{RP}})$ verifies the associated RP authentication request auth_{RP} and access token τ_{ac} , and generates shares of a threshold signature sigshare_i on the previously disclosed attributes $\vec{m}_d$ for each $i = 1, 2, \dots, N$. It returns an identity token share $\tau_{id, [i]} = \{\vec{m}_d, \text{sigshare}_i\}$ for each i .
- $\mathcal{F}_{\text{TIDT}}^{(t, N)}.\text{aggregate}(\{\tau_{id, [i]}\}_{i \in \text{SSet}})$ aggregates t identity token shares, namely $\{\tau_{id, [i]}\}_{i \in \text{SSet}}$, where SSet is the set of the t share identifiers. It constructs the threshold signature σ_T from the t sigshare 's. It verifies the disclosed attributes $\vec{m}_d$ associated with σ_T with the corresponding public key and returns a success or failure response.

In the context of anonymous credential issuance, the central issuer can be a single point of failure if it becomes unavailable or corrupt. To tackle this limitation, Doerner et al. [43] thresholdize the signing phase of the BBS+ signature scheme [33].² This threshold BBS+ signature scheme requires a group of signers to participate in a multiparty computation (MPC) process to generate signature shares. We adopt this scheme for thresholdizing identity token generation, thus realizing the ideal functionality $\mathcal{F}_{\text{TIDT}}^{(t, N)}$, with the ASs serving as MPC participants. This allows RPs to continue using their legacy signature verification routine once aggregating the signature

²BBS+ slightly modifies the BBS signature composition so that it is proved secure under the q-SDH assumption.

shares while supporting the efficient delivery of user identity information. We provide more details on this scheme in Appendix E.

D. Verifiable Data Registry

Definition 4 (Ideal Functionality $\mathcal{F}_{\text{VDR}}$). *The ideal verifiable data registry functionality $\mathcal{F}_{\text{VDR}}$ provides the following interfaces for accessing the VDR:*

- $\mathcal{F}_{\text{VDR}}.\text{write}(\text{meta}_{\text{RP}}, \text{sch}_{\text{VP}}, \text{rid})$ updates the TAL of the IdP and records an RP registration. It records RP's metadata meta_{RP} and schema definition sch_{VP} against the IdP assigned RP's identity rid in the ASs accessible VDR.
- $\mathcal{F}_{\text{VDR}}.\text{read}(\text{rid})$ fetches the registered RP's schema definitions and SSO service terms from the VDR using the RP's identity rid . It returns SUCCESS if the read is successful; FAIL otherwise.

In the decentralized identity ecosystem, a VDR provides vital functionalities for creating and managing VCs, distributing PKI information, data sharing, and identity management services [44], [13], for which we formulate as the ideal functionality $\mathcal{F}_{\text{VDR}}$. It acts as a trusted intermediary between the various entities of the VeriSSO system model. In VeriSSO's RP registration phase, the IdP records the RP's service endpoints, public-key certificates, and schema definitions, which are used by ASs and users in the authentication phase to prevent users from accessing fraudulent and phishing websites.

V. VERISSO DETAILED DESIGN

We describe the detailed VeriSSO design. We begin with the setup phase, which consists of RP registration and user credential issuance. Next, we present the authentication phase, during which registered users gain access to RP services.

A. RP Registration

VeriSSO requires RPs to register their identity with the IdP, similar to OIDC. Fig. 3 shows the RP registration protocol. Specifically, the IdP runs the following wrapper function:

- **RegisterRP**(req): parsing the RP registration request req into $(\text{id}_{\text{RP}}, \text{meta}_{\text{RP}}, \text{sch}_{\text{VP}})$, where meta_{RP} contains the RP's service endpoint url_{RP} and sch_{VP} is the RP's required VP attribute schema (i.e., the list of user attributes required for log-in). Next, the IdP assigns a unique identifier rid to the RP and uploads the RP's metadata into its TAL on the VDR using the $\mathcal{F}_{\text{VDR}}.\text{write}$ interface, before returning rid and meta_{IdP} (the IdP's metadata) to the RP as res .

Compared to OIDC's RP registration protocol [36], VeriSSO does not modify RP-side operations but requires involvement with the VDR. The RP identifier rid assigned by the IdP will be used by ASs to fetch sch_{VP} from the VDR in the authentication phase.

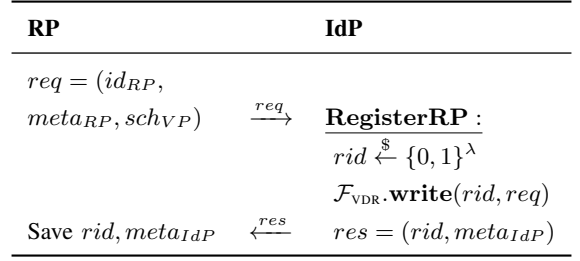


Fig. 3: VeriSSO RP Registration Protocol.

B. User Credential Issuance

A user, as a registered client of an IdP, requests a VC from the IdP. The user selects a set of attributes (e.g., profession, institution, student status, zip code, etc.), denoted $\vec{m} = (m_1, m_2, \dots, m_L)$ and a random scalar u that binds the credential to the user's device. The user-side secret u remains hidden from the IdP and allows the user to generate pseudonyms in the authentication phase. Fig. 4 shows the workflow involved in the credential request and signing process using the ideal functionality $\mathcal{F}_{\text{BIVC}}$ interfaces. We describe the credential request protocol using the following wrapper function:

- **RequestVC**($\cdot$): The user selects the list of attributes/claims $\vec{m}$ and a random device (where the wallet resides) binding secret u . A blinding factor d is used to blind u into $u' = u^d$. Then, a partially-blind credential issuance request consisting of $\vec{m}$ and u' is submitted to the IdP's endpoint. The wallet is expected to receive a VC cred , a blind signature $\hat{\sigma}$ of the VC, and an IdP-assigned unique credential identifier γ (included in cred). The cred generation is shown next.

The IdP is responsible for issuing VCs to users by attesting a set of requested attributes in $\vec{m}$. It also ensures that each VC includes a unique identifier γ that acts as a backdoor mechanism for accountability of dishonest users. The user can hide the identifier during the authentication phase to preserve privacy and anonymity. We describe credential issuance using the following function:

- **IssueVC**(req) $\rightarrow \hat{\text{cred}}$: The IdP assigns a unique credential identifier γ , and drafts a W3C-standardized VC, cred , with the user's claims $\vec{m}$, a blind user secret u' , and γ as the subject attributes. Then the IdP produces a blind signature $\hat{\sigma}$ on the combined vector of attributes $(\vec{m}, u', \gamma)$ using the BBS signing algorithm (BBS.sign). The IdP then appends some auxiliary metadata and finalizes the partially-blind VC as:

$$\hat{\text{cred}} = (\vec{m}, u', \gamma, \hat{\sigma}) \quad (1)$$

The IdP also logs the VC issuance internally for future audit/accountability purposes.

To wrap up **RequestVC**, upon receiving $\hat{\text{cred}}$, the user unblinds it using the $\mathcal{F}_{\text{BIVC}}.\text{unblind}$ interface with the previously selected blinding factor d to derive σ and thus the VC cred :

$$\text{cred} = (\vec{m}, u, \gamma, \sigma) \quad (2)$$

User	IdP
RequestVC :	
$u, d \xleftarrow{\$} \{0, 1\}^\lambda$	
$\vec{m} = (m_1, \dots, m_L)$	
$\mathcal{F}_{\text{BIVC}}.\text{prep}(\vec{m}, u, d)$	
$\rightarrow (\vec{m}, u')$	$\xrightarrow{\vec{m}, u'}$
	IssueVC :
	$\gamma \xleftarrow{\$} \{0, 1\}^\lambda$
	$\mathcal{F}_{\text{BIVC}}.\text{issue}(\vec{m}, u', \gamma)$
$\mathcal{F}_{\text{BIVC}}.\text{unblind}(\hat{cred}, d)$	$\xleftarrow{\hat{cred}}$
$\rightarrow cred = (\vec{m}, u, \gamma, \sigma)$	$\hat{cred} = (\vec{m}, u', \gamma, \hat{\sigma})$

Fig. 4: VeriSSO User Credential Issuance Protocol.

C. Authentication Phase

The VeriSSO authentication phase is composed of five consecutive stages: initiation, RP identification, user verification, and identity token issuance. The complete workflow is shown in Fig. 5, with each important step enclosed in a function block. Next, we describe these stages and the involved protocol functions in more detail.

1) *Initiation*: This SSO authentication is initialized by the user and marks the start of an SSO authentication session. Specifically, the user accesses the designated login web address/endpoint of an RP, informs the RP about its login preferences (the user's IdP), and receives an authentication request that specifies the RP's requirements. It involves the user invoking the RP's **InitAuth** procedure with the user's IdP's metadata $meta_{\text{IdP}}$, which returns an authentication request $auth_{\text{RP}}$ that binds the corresponding IdP-specific identity rid (obtained at the RP registration phase, discussed previously in section V-A) and schema definition sch_{VP} to the user's authentication session.

2) *RP Identification*: This stage is executed between the user, who finished the initiation stage, and individual ASs. The user forwards the SSO authentication request and RP's requirements to each AS, who verify the RP's registration and service terms with the VDR. The user waits for at least a threshold t number of positive responses before it is proceeds to the next stage. We describe the interaction between the user and each AS using the following function:

- **IdentifyRP**($auth_{\text{RP}}, sch_{\text{VP}}$) $\rightarrow \{\text{APPROVE}, \text{DENY}\}$: To verify the authorization of the RP and its membership agreement with the IdP, the user queries the ASs to confirm the authentication request $auth_{\text{RP}}$ and attribute requirements schema sch_{VP} from the TAL of the IdP. Individual ASs query the VDR that implements the $\mathcal{F}_{\text{VDR}}.\text{read}$ functionality to fetch the corresponding TAL record for rid .

When the user receives a threshold t number of positive responses from the ASs, the RP is considered to be verified.

3) *User Verification*: After verifying RP's authentication request and service requirements with the TAL records, the user presents a pseudonymous, partially-disclosed VP to identify itself to the ASs, who verify the validity of the VP and

the disclosed attributes according to the RP's authentication request. The ASs then return access tokens to the user who forwards them to the RP, to signal the completion of the user verification phase. The access token acts as the key to the disclosed user attributes, which are temporarily held by the ASs for the duration of the corresponding access token expiry. This stage involves the realization of $\mathcal{F}_{\text{SDVP}}$ interfaces with the following sub-routines executed between the user and ASs:

- **PrepareVP**($cred, auth_{\text{RP}}, sch_{\text{VP}}, domain_{\text{RP}}$) $\rightarrow pres$: First, The user hides the permanent VC identifier γ and user secret u and generates a pseudonym for each RP. In this process, γ is encrypted with the ElGamal [45] ciphertext $E(\gamma)$ with the public key y , u is used to produce an RP-specific pseudonym $\tilde{u}$ based on the RP's domain name $domain_{\text{RP}}$. The user then computes a ZKP ϕ , of his knowledge of the identifiers (γ, u) the correctness of pseudonym generation (we discuss the details in Appendix D2).
Next, the user reviews the attribute requirements (from the RP's schema definition sch_{VP}) and selects the attributes to disclose $\vec{m}_d$, and hides the remaining set $\vec{m}_h$. Then the user computes an *SPoK* π that proves the knowledge of the original $cred$ signature and the correctness of the selective disclosure of attributes.
Finally, a pseudonymous selectively disclosed VP $pres$ is constructed as:

$$pres = (\vec{m}_d, E(\gamma), \tilde{u}, \phi, \pi) \quad (3)$$

- **VerifyVP**($pres, auth_{\text{RP}}$) $\rightarrow \tau_{ac}$: Once an AS receives the VP $pres$, the validity of its original VC $cred$ and undisclosed attributes can be verified according *SPoK* verification algorithm discussed in Appendix D1. In addition, this step also involves verifying the ElGamal ciphertext $E(\gamma)$, pseudonym $\tilde{u}$, and the ZKP ϕ as part of the user verification process.

When $pres$ construction and attribute set $\vec{m}_d$ are verified, each AS returns its own access token τ_{ac} that encapsulates the $pres$ verification results with the authentication request $auth_{\text{RP}}$. The user accumulates at least a threshold t number of τ_{ac} 's and relays them to RP.

4) *Identity Token Issuance and Verification*: When the RP obtains at least t number of access tokens from the user, it is convinced of a successful user verification. Next, the RP establishes a secure back channel with the ASs, to verify its service authorization and requests them to release the user attributes disclosed in the VP by posting the access token and the associated authentication request to each AS. At this stage, if the user has previously visited the same RP, it may use the pseudonym $\tilde{u}$ to lookup the user's existing profile and finish the SSO protocol without the identity token issuance routine.

- **SignOnToken**($\tau_{ac}, auth_{\text{RP}}$): The RP receives access token τ_{ac} and the corresponding authentication request $auth_{\text{RP}}$ from the user. If the user forwards the threshold t number of access tokens, the RP is convinced of the user's identity verification by the ASs.
- **IssueIdToken**($auth_{\text{RP}}, \tau_{ac}$) $\rightarrow \tau_{id}$: The AS committee receives the associated authentication request $auth_{\text{RP}}$

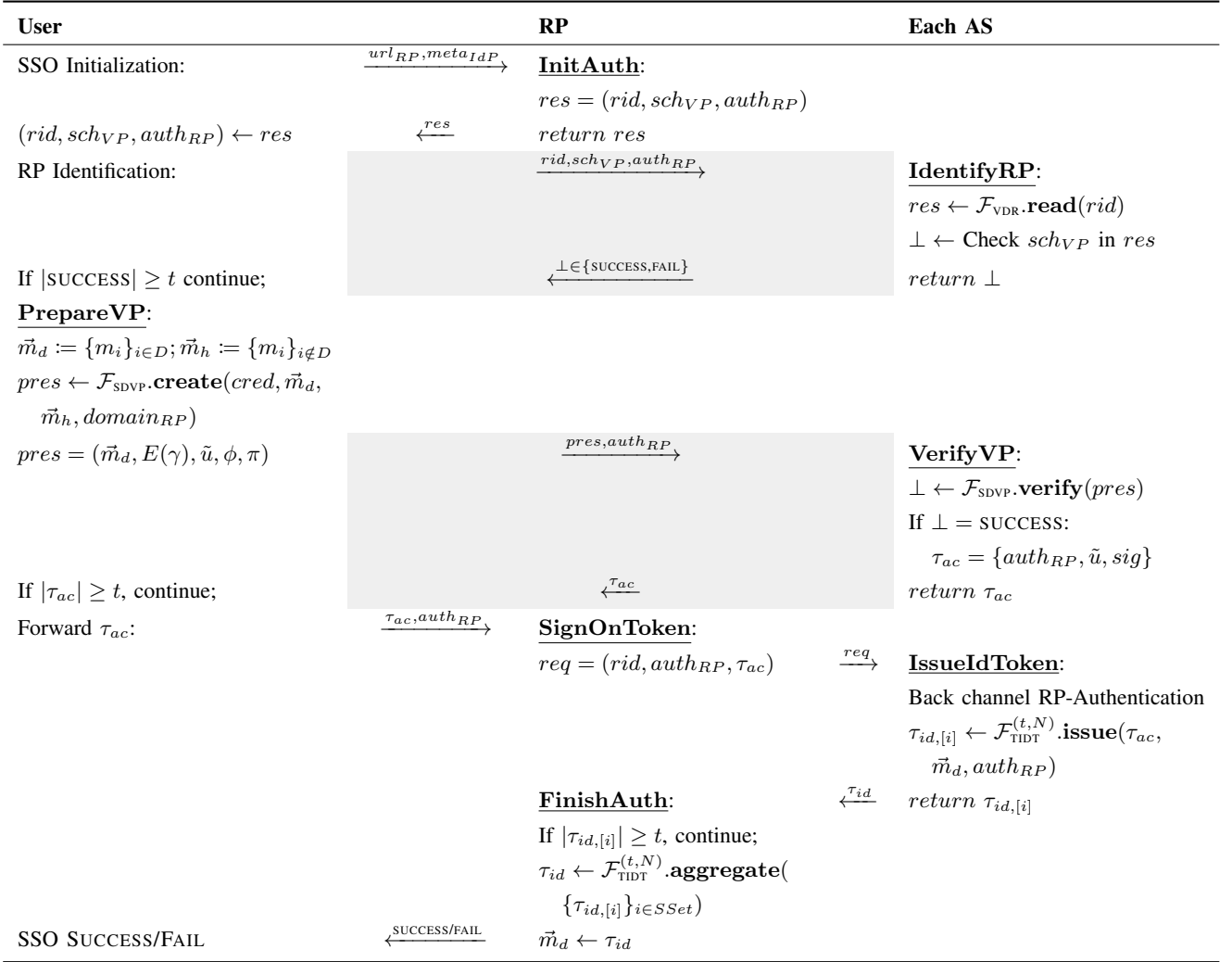


Fig. 5: VeriSSO Authentication Protocol. $\perp$: Function call response as SUCCESS/FAIL. $|X|$: Number of responses of X from all ASs. : User to AS communication. *sig*: Standard PKI signature.

and access token τ_{ac} , and establishes a secure back channel to authenticate the RP (using standard TLS). The ASs engage in the (t, N) -threshold protocol as defined in $\mathcal{F}_{TIDT}^{(t, N)}$.*issue* to generate a token consisting of the previously disclosed user attributes $\vec{m}_d$ from the VP *pres*, and compute shares of a threshold signature. We call this token $\tau_{id, [i]} = \{\vec{m}_d, sigshare_i\}$.

- **FinishAuth**($\vec{m}_d, \{\tau_{id, [i]}\}_{i \in SSet}$): From the signature shares ($\{sigshare_i\}_{i \in N}$) where N is the set of responding ASs ($SSet$), and when t responses are received, the RP can reconstruct the signature of the identity token τ_{id} using $\mathcal{F}_{TIDT}^{(t, N)}$.*aggregate*. The RP verifies the aggregated signature with the AS committee's threshold public key (stored on the VDR). If verified, the RP grants access to the user and concludes the SSO session.

D. Lawful De-anonymization

During the sign-on process, the user prepares an RP-specific pseudonym $\tilde{u}$ to uniquely identify itself to an RP. If an anonymous VP must be de-anonymized for accountability,

the AS committee can reveal the associated user. Certain AS members with lawful authorities (i.e., the decryption authorities) collectively use their private key (corresponding to the combined encryption key y) to decrypt the VC identifier ($ENC_y(\gamma)$) attached in *pres*. The AS committee will thus recover γ which can be traced back to the IdP assigned permanent identifier γ and to the user's VC. We describe the de-anonymization routine using the following function:

- **RevealUser**(*pres*) $\rightarrow \gamma$: The ASs retrieve the encrypted permanent identifier $E(\gamma)$ and the RP-specific pseudonym $\tilde{u}$. Upon receiving a de-anonymization request, the ASs with lawful decryption privileges aggregate the decryption key corresponding to y and reveal γ . The IdP then cooperates with the ASs to fetch the data related to γ and the corresponding VC *cred* from its internal database, essentially uncovering the real identity of the user. Subsequently, an alert may be issued to the ASs and RPs for appropriate actions.

VI. ANALYSIS

A. Security Analysis

Under the assumptions outlined in section III-B, we will show VeriSSO satisfies the security and privacy goals discussed in section III-C. On a high level, the security and privacy of VeriSSO boils down to that of the security assumptions of BBS signature scheme [46], [43] and zero-knowledge proofs. In the registration phase, the user-side secret u and the IdP-assigned permanent identifier γ are kept secret with the respective entities. In the authentication phase, the user hides γ , u , the credential signature σ , and presents to ASs a ZKP of their knowledge and possession in the *pres*. Moreover, the user derives a new pseudonym $\tilde{u}$ to produce a unique identity for each RP.

Lemma 1 (Unforgeability of VC/VP). *In VeriSSO, the VC signing/issuance process in the user registration phase (see section V-B) and the VP generation and verification algorithms in the user authentication phase (see section V-C) are unforgeable for all probabilistic polynomial time (PPT) adversaries $\mathcal{A}$ except for a negligible probability under the q -Strong Diffie-Hellman (q -SDH) assumption (see Appendix F). Specifically, given a ZKP π that comes with a VP *pres*, $\mathcal{A}$ can generate other valid VPs only if the complete set of attributes $\vec{m}$, user secret u , and permanent identifier γ are known.*

Proof. The unforgeability property of VC *cred* can be derived intuitively from the unforgeability of the BBS signature scheme [32], [46], which is based on the q -SDH assumption. It ensures that an adversary $\mathcal{A}$ cannot forge a legitimate credential *cred* or a valid proof of a valid attribute set $\vec{m}$ signed by the issuer's signing key sk .

As for the unforgeability property of VP *pres*, the VP *pres* in VeriSSO comprises the ZKP of the issuer's signature σ (known only to the user and IdP), IdP assigned permanent identifier γ (known only to the user and IdP), and user secret u (known only to the user). Forging a *pres* would thus require full control of the *cred* holder.

Lastly, the (t, N) threshold identity token τ_{id} signature produced by $\mathcal{F}_{BBS+}(n, t, \mathbb{G})$ is secure even if an adversary $\mathcal{A}$ statistically corrupts up to $N - t$ number of ASs. We refer to [43] for the formal security proof and analysis of $\mathcal{F}_{BBS+}(n, t, \mathbb{G})$. $\square$

Theorem 1 (Correctness of user authentication in VeriSSO). *If the q -SDH Assumption holds, the ZKP system satisfies completeness, soundness, and zero-knowledge properties, and at most $(N - t)$ ASs are compromised (not functioning honestly), then VeriSSO guarantees the success of the user authentication protocol, given that it has produced the *pres* from a valid *cred* as described in Lemma 1.*

Proof. The correctness of our SSO scheme builds on its composing protocols, namely the BBS signature-based VC scheme [47], [46], the threshold BBS signature [43] for ID token generation, and the public key signature scheme for verifying RP authenticity. The BBS-based VC scheme signature is secure under the q -SDH assumption, shown in the above lemma. This means only users with valid IdP registration

can generate a valid VP for AS to verify. As for identity token generation, the threshold BBS scheme $\mathcal{F}_{BBS+}(n, t, \mathbb{G})$ also produces a BBS signature, whose unforgeability follows the above lemma and the t -threshold assumption [43]. This ensures that the identity token can be aggregated, whose BBS signature can be verified by the RP. This essentially binds VP-based user verification with the threshold issuance of the identity token.

Lastly, the correctness of lawful user de-anonymization, if needed, is based on the validity of the ElGamal encryption of the user's credential identifier γ , which, in turn, is provided by the completeness and soundness of ZKP. $\square$

Next, we prove the privacy properties of VeriSSO, using a game-based proof framework. We aim to construct a *challenger* that can simulate VeriSSO protocol execution under different scenarios of adversarial control over the system entities, and then show that the adversary ($\mathcal{A}$) can only succeed with a negligible probability.

Lemma 2 (Minimal Disclosure and Anonymity). *VeriSSO allows users only to disclose attributes required by RP's schema (sch_{VP}) to pass authentication, while concealing other necessary attributes from any RP or AS.*

Proof. The selectively disclosed *pres* does not reveal any permanent user identifier (u, γ) , except the required attributes, ZKP, and the RP-specific pseudonym $\tilde{u}$, which is not associated with any long-term identity. Since ASs only see the *pres* and RPs receive identity tokens from ASs, curious ASs or RPs cannot discover the long-term user identifier except RP-specific pseudonyms. $\square$

Next, we show VeriSSO unlinkability properties using three unlinkability games, as defined in Fig. 6, Fig. 7, Fig. 8, respectively.

$G_{IdP-Unlinkability}^{VeriSSO}(1^\lambda, \mathcal{A})$
1. Initialize VeriSSO system and functionalities. 2. $IdP \leftarrow \mathcal{A}$. 3. Simulate two RPs RP_0 and RP_1. 4. Simulate a user U_0 with identifiers (u_0, γ_0), and register with IdP. // $\mathcal{A}$ learns γ_0 5. $b \xleftarrow{\\$} \{0, 1\}$. 6. U_0 logs in to RP_b following VeriSSO user authentication protocol. 7. $b' \leftarrow \mathcal{A}()$ 8. return 'success' if $b = b'$; 'fail' otherwise.

Fig. 6: IdP Unlinkability Game.

Theorem 2 (IdP Unlinkability). *Under the game $G_{IdP-Unlinkability}^{VeriSSO}$ as described in Fig. 6, a PPT adversary $\mathcal{A}$ succeeds with chance no more than $1/2 + \text{negl}(\lambda)$. In other words, any adversary who controls the IdP cannot link two VeriSSO authentication sessions to a specific user.*

$G_{RP-Unlinkability}^{VeriSSO}(1^\lambda, \mathcal{A})$
1. Initialize VeriSSO system and functionalities. 2. $RP_0, RP_1 \leftarrow \mathcal{A}$. 3. Simulate two users U_0, U_1 with identifiers (u_0, γ_0) and (u_1, γ_1), and register with IdP. 4. U_0 and U_1 login to RP_0 following VeriSSO user authentication protocol. 5. RP_0 follows $\mathcal{F}_{TIDT}^{(t,N)}$ to retrieve the disclosed attributes and identity tokens. // $\mathcal{A}$ learns pseudonyms $u_{0,0}, u_{1,0}$ 6. $b \xleftarrow{\\$} \{0, 1\}$. 7. U_b login to RP_1 following VeriSSO user authentication protocol. 8. $b' \leftarrow \mathcal{A}(u_{b,1})$ 9. return ‘success’ if $b = b'$; ‘fail’ otherwise.

Fig. 7: RP Unlinkability Game

$G_{Unlinkability}^{VeriSSO}(1^\lambda, \mathcal{A})$
1. Initialize VeriSSO system and functionalities. 2. $(t-1)$ AS's, $RP_0, RP_1 \leftarrow \mathcal{A}$. 3. Simulate two users U_0, U_1 with identifiers (u_0, γ_0) and (u_1, γ_1), and register with IdP. 4. U_0 and U_1 login to RP_0 following VeriSSO user authentication protocol. 5. RP_0 follows $\mathcal{F}_{TIDT}^{(t,N)}$ to retrieve the disclosed attributes and identity tokens. // $\mathcal{A}$ learns pseudonyms $u_{0,0}, u_{1,0}$ 6. $b \xleftarrow{\\$} \{0, 1\}$. 7. U_b login to RP_1 following VeriSSO user authentication protocol. 8. $b' \leftarrow \mathcal{A}(u_{b,1})$ 9. return ‘success’ if $b = b'$; ‘fail’ otherwise.

Fig. 8: Collusive AS-RP Unlinkability Game

Proof. Despite that $\mathcal{A}$ can learn the IdP-assigned identifier γ_0 when user U_0 registers with the IdP and the two RPs’ identity; however, since the IdP is never involved in any of the routines during the user authentication phase, $\mathcal{A}$ is unable to determine any linkage between γ_0 and any RP. Therefore, determining which of RP_0 and RP_1 is accessed by U_0 is no different from random guessing. $\square$

Theorem 3 (Collusive-AS-RP Unlinkability). *Under the game $G_{Unlinkability}^{VeriSSO}$ as described in Fig. 8, a PPT adversary $\mathcal{A}$ succeeds with chance no more than $1/2 + \text{negl}(\lambda)$. In other words, any adversary who controls any number of RPs and at most $(t-1)$ ASs cannot link two VeriSSO authentication sessions with different RPs to a specific user.*

Proof. The challenger registers two simulated users U_0 and U_1 with the IdP. Subsequently, the two users follow the VeriSSO

authentication protocol to access RP_0 , where $\mathcal{A}$ learns their respective pseudonyms $u_{0,0}$ and $u_{1,0}$. Next, when a random user accesses RP_1 , we calculate $\mathcal{A}$ ’s chance of identifying the associated user.

From the above $G_{IdP-Unlinkability}^{VeriSSO}$ and $G_{Unlinkability}^{VeriSSO}$ games, we demonstrate that given $(\gamma_0, u_{0,0}), (\gamma_1, u_{1,0})$ and $u_{b,1}$, adversary $\mathcal{A}$ cannot determine b , where γ_i is the VC identifier for U_i , $u_{i,0}$ is the pseudonym for U_i at RP_0 and $i \in \{0, 1\}$. We prove the games with the help of a hybrid system. We assume there is a hypothetical system $VeriSSO'$ that uses a random function $\text{rand}()$ to compute $\tilde{u}$. We also assume an abstraction of the original $\tilde{u}$ generation subroutine in VeriSSO denoted as $RPPF$ (RP-Specific Pseudonym Function) that takes the arguments $(\text{domain}_{RP}, \gamma)$ as input and simulates the computation of $\tilde{u}$.

We argue that if $\tilde{u}$ is generated using a random function, the probability of $\mathcal{A}$ winning the game is equivalent to bit guessing, that is $\Pr[G_{Unlinkability}^{VeriSSO} \Rightarrow 1] = 1/2$. In other words, we show that $\mathcal{A}$ cannot distinguish between the real world $G_{Unlinkability}^{VeriSSO}$ and the hypothetical $G_{Unlinkability}^{VeriSSO'}$. To prove the indistinguishability, we construct an adversary $\mathcal{B}$ with the same advantage as $\mathcal{A}$ for the $RPPF$ game described in Fig. 11. Let’s say, the advantage of $\mathcal{A}$ being able to distinguish between $G_{Unlinkability}^{VeriSSO}$ and $G_{Unlinkability}^{VeriSSO'}$ is given by $\text{Adv}_{\mathcal{A}}(\lambda) = |\Pr[G_{Unlinkability}^{VeriSSO} \Rightarrow 1] - \Pr[G_{Unlinkability}^{VeriSSO'} \Rightarrow 1]|$.

In the $RPPF$ game, adversary $\mathcal{B}$ receives a sequence of queries $x_1, x_2, \dots$ from $\mathcal{A}$ and submits them to the challenger, where $x_i := (\text{domain}_{RP}, \gamma_i)$ for $i = 1, 2, \dots$. The challenger randomly selects a function f to compute $y_i \leftarrow f(x_i)$, and returns the results to $\mathcal{B}$. Subsequently, $\mathcal{B}$ forwards the sequence of y_i to $\mathcal{A}$ and pretends each y_i as $\tilde{u}_i$. Finally, when $\mathcal{A}$ returns its guess in $G_{Unlinkability}^{VeriSSO}$ game, $\mathcal{B}$ provides the same guess to the challenger as its own output.

let E_b be the guess of $\mathcal{B}$ in the $RPPF$ game for $b \in \{0, 1\}$. Then the advantage of $\mathcal{B}$ is given by $\text{Adv}_{\mathcal{B}}^{RPPF}(\lambda) = |\Pr[E_0] - \Pr[E_1]|$. From the the reduction game (Fig. 12) we can derive that $\Pr[E_0] = \Pr[G_{Unlinkability}^{VeriSSO} \Rightarrow 1]$ and $\Pr[E_1] = \Pr[G_{Unlinkability}^{VeriSSO'} \Rightarrow 1]$. This gives $\text{Adv}_{\mathcal{B}}^{RPPF}(\lambda) = \text{Adv}_{\mathcal{A}}(\lambda)$. Thus, we can say that $\mathcal{B}$ has the same advantage as $\mathcal{A}$. Also, $\mathcal{A}$ cannot distinguish between $G_{Unlinkability}^{VeriSSO}$ and $G_{Unlinkability}^{VeriSSO'}$, except for a negligible probability. Given that $\Pr[G_{Unlinkability}^{VeriSSO}(\lambda, \mathcal{A}) \Rightarrow 1] = 1/2$, we can say $\Pr[G_{Unlinkability}^{VeriSSO}(\lambda, \mathcal{A}) \Rightarrow 1] \leq 1/2 + \text{negl}(\lambda)$. $\square$

Corollary 1 (RP Unlinkability). *Under the game $G_{RP-Unlinkability}^{VeriSSO}$ as described in Fig. 7, a PPT adversary $\mathcal{A}$ succeeds with chance no more than $1/2 + \text{negl}(\lambda)$. In other words, any adversary who controls any number of RPs cannot link two VeriSSO authentication sessions with different RPs to a specific user.*

Proof. This result follows naturally from Theorem 3. $\square$

Despite the unlinkability property, next show that an RP can still associate two SSO sessions if they are initiated by the same anonymous user, which is crucial for Sybil resistance.

Corollary 2 (Intra-RP Linkability). *An RP can correlate SSO sessions to the same anonymous user registered under the same IdP.*

Proof. If two VPs are for the same RP, $\mathcal{A}$ sees the RP-specific pseudonym $\tilde{u}$, which is used for intra-RP linkability (for RP’s in-house profile management). However, $\tilde{u}$ does not reveal any long-term identity information (see the above lemma). Therefore, the VPs are unlinkable to a specific user from the view of a curious AS or RP. $\square$

Finally, we show that the minimal disclosure and unlinkability properties hold even the IdP, RP, and $N - t$ ASs collude.

Corollary 3 (Correctness of Lawful De-anonymization). *VeriSSO authentication protocol requires users to provide a threshold encryption of their IdP-assigned VC identifier γ and prove in zero knowledge that it is well-formed, allowing the ASs to verify the user’s identity without decrypting it. This construction also allows ASs with decryption privileges to reveal the user’s identity if a user misuses the anonymity.*

Proof. In the user authentication phase, the user is required to encrypt the IdP-assigned identifier using the threshold public key of the decryption authorities and attach a ZKP of its possession. The correctness of this construction and thereby decryption for accountability is inherently derived from the correctness of the threshold version of ElGamal encryption [48], which is based on the Decisional Diffie-Hellman (DDH) security assumption [49], [50]. $\square$

B. Discussion on Practical Deployment

Who will be ASs? Using third-party authentication services for access control has been quite popular in the commercial space, represented Auth0 [34] and OneLogin [35]. In VeriSSO, we envision that these commercial authentication services can be transitioned into the ASs, given that they can communicate for joint verification tasks. New candidates for the authentication committee include delegates of well-known IdPs and RPs. The profit model and incentive mechanisms of authentication servers are an important but orthogonal research effort.

Who will be the VDR? In a typical distributed identity management system, the services of a VDR are typically provided by blockchains or any distributed database [22], [13], [51], [37], [44]. Their services include access control, persistent data storage, and PKI. VDR can also be implemented using Information Centric Networking (ICN) technology, where data is stored on ICN register nodes and API calls to the VDR are translated into ICN flows [52].

Multi-device support. According to the issuance phase of VeriSSO, every credential is linked to a wallet/device through a permanent credential identifier γ assigned by the IdP. This allows the user to prove his previous registration and obtain credentials on multiple wallets. The user-side secret u also enables the IdP to track the issued credentials.

Support for Multi-Factor Authentication. VeriSSO is an authentication protocol leveraging the single factor of credential ownership. However, an MFA module could be easily integrated to offer multiple levels of security using standard

protocols like FIDO [53]. We consider MFA support to be an implementation design decision and VeriSSO architecture can accommodate such an extension while preserving its privacy and usability properties.

Supporting multiple IdPs. Some RPs may require users to prove attributes from different IdPs, for instance, to show diploma proof issued by a university and completion certificates issued by an online course provider. The W3C standard’s VC and VP data model we used intrinsically supports multiple issuers [54] and allows a user to compose a VP using multiple VCs.

C. Discussion on Limitations

Linkage through external data. In our system, although VPs derived during the authentication phase are pseudonymous and unlinkable through a specific identifier, it is possible that an attacker may find correlation between attributes disclosed by multiple VPs and user data from external domains, e.g., social media websites, and infer user identity through data analysis [55]. We consider this to be an important data anonymization challenge for future work.

Credential revocation. VeriSSO currently provides a coarse form of credential revocation, i.e., in the event of wallet theft or compromise of a device, the user can instruct the IdP to terminate the associated credentials and publish an alert to the VDR. However, incorporating a fine-grained credential revocation scheme, such as [56], is feasible if it does not significantly increase user-side overhead.

VII. IMPLEMENTATION AND EVALUATION

A. Prototype Implementation

We implemented a prototype to realize VeriSSO’s core functionalities with approximately 1872 lines of Rust code.³ Our implementation of BBS signature uses BLS12-381 Elliptic curve [57]. The various core subroutines of VeriSSO use the Rust library `bbs_plus` version 0.23.0 [58] to implement the standard and MPC versions of the signature scheme according to the protocol discussed in Section IV.

Correctness Verification with ProVerif. We also provide an abstract protocol verification of VeriSSO using ProVerif [59].⁴ The tool operates with an abstract symbolic representation of cryptography, assuming the Dolev–Yao adversary [60]. Protocol messages are represented as terms, cryptographic primitives are modeled as function symbols assumed to be ideal, and their algebraic properties are captured through equational theories. The protocol behavior is described as processes, which are further encoded into horn clauses for automated reasoning. Security goals such as secrecy and correctness are formalized as reachability or correspondence queries, reducing the analysis to a horn clause unification problem [61]. VeriSSO ProVerif model verifies the user registration (VC creation and signing) and user authentication (VP creation and threshold token signing) phases with black-box cryptographic building blocks.

³Code and artifacts: <https://anonymous.4open.science/r/verisso-7E17/>.

⁴Artifact available under the `proverif` folder from the above repository.

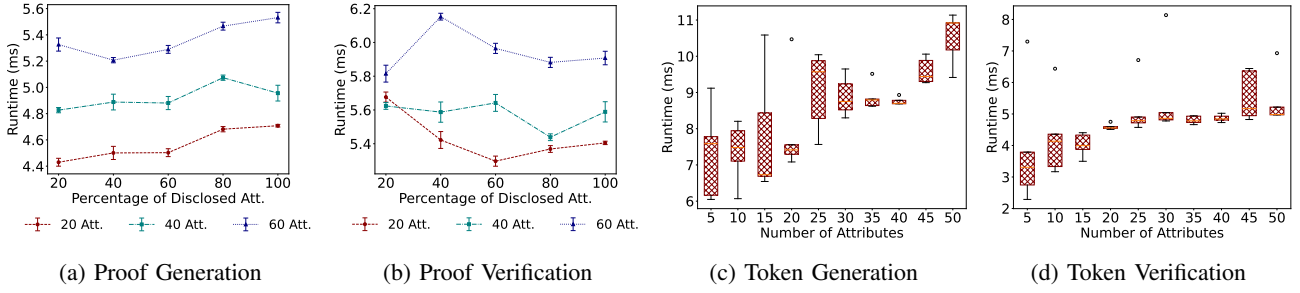


Fig. 9: Runtime performance of key VeriSSO operations.

Our experimental setup includes a server for simulating IdP functionalities for the setup and user registration phase. In the authentication phase, the user interacts with up to 8 AS nodes running verification routines and managing threshold tokens (with a threshold requirement of 5 nodes). We deploy our experimental setup with 8 individual nodes in CloudLab. Each node is equipped with an Xeon E5-2450 processor (8 cores, 2.1 GHz), 16 GB of RAM, and Ubuntu 22.04.2 LTS operating system.

B. Benchmarking

We only show the runtime results of the authentication phase, as the registration phase is a one-time setup and can be completed asynchronously.

User Verification. Figures 9a and 9b show the runtime of *pres* proof generation and verification, respectively, under varying *cred* attribute sizes (20, 40, and 60 attributes) and disclosure percentages (20%-100%). In both cases, the results indicate that performance is determined primarily by the total number of attributes rather than the disclosure ratio. For *cred* proof generation, runtime increases only slightly with attribute size, ranging from ≈ 4.4 – 4.7 ms for 20 attributes to ≈ 5.1 – 5.4 ms for 60 attributes, with minimal sensitivity to the proportion disclosed. *pres* proof verification follows a similar pattern, requiring ≈ 5.3 – 5.6 ms for 20 attributes, ≈ 5.4 – 5.7 ms for 40 attributes, and ≈ 5.8 – 6.1 ms for 60 attributes, again with only minor fluctuation across disclosure levels. Overall, across all tested configurations, both proof generation and verification consistently complete within ≈ 4.4 – 6.1 ms.

Threshold ID Token Issuance, Aggregation, Verification.

Figures 9c and 9d show the runtime latency of token generation and verification as the number of attributes in the user identity token increases from 5 to 50. Token generation exhibits a steady increase from ≈ 6 – 7 ms at 5 attributes to ≈ 10 – 11 ms at 50 attributes, with slightly larger variance at higher counts. Token verification follows a similar upward trend but incurs lower costs, rising from ≈ 2 – 3 ms to ≈ 5 – 6 ms over the same range, with only limited variability except for a few outliers. Overall, both operations scale linearly with the number of attributes, with generation consistently more expensive than verification, yet both were found to be quite efficient (< 12 ms) even at 50 attributes.

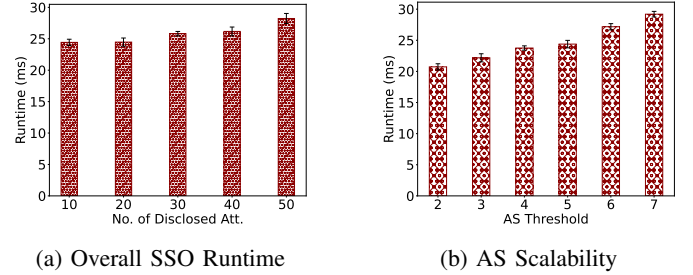


Fig. 10: Comparison of the execution time of one full SSO session.

C. Overall Runtime

Figure 10 reports the start-to-end latency of a complete SSO session across two workload dimensions: the number of disclosed attributes and the required AS threshold. In the overall SSO runtime experiment (Figure 10a), we fix the total attribute pool at 60 and vary the number of disclosed attributes from 10 to 50; the runtime remains essentially flat, increasing only from about 25 ms to just under 29 ms. In the AS scalability experiment (Figure 10b), we fix the disclosed attribute set size at 5 and vary the threshold requirement of signers from 2 to 7; latency grows monotonically and roughly linearly from about 21 ms to 30 ms, consistent with the additional shares/signatures and verifications, yet stays below 30 ms even at the highest threshold, preserving responsiveness. Overall, a full SSO session completes in roughly 20–30 ms in all cases, remains well stable with more disclosed attributes, and adds only a small extra cost as the threshold requirement increases.

We also evaluated the overall SSO runtime of EL PASSO [11], a credential-based SSO scheme similar to ours, under identical conditions. We observe a similar increase in runtime with the growing number of attributes. Notably, when disclosing 50 out of 60 attributes, VeriSSO is found to be slightly costlier (approx. 10 ms), primarily because EL PASSO is based on the Implicit Flow that requires only two rounds of interaction between the user and RP and does not support RP authentication. In contrast, VeriSSO supports the ACF sequence (supporting RP authentication) and involves ASs to verify VP and issue ID tokens that require three extra rounds of communication.

TABLE II: Comparison of Relevant SSO Solutions.

Related SSO Solutions \ Properties	No single-point failure	Inter-RP unlinkability	Selective disclosure	Intra-RP linkability	Lawful de-anonymiz.	Multi-device support	RP authenticat.	Compatible with ACF
Original OIDC [1]	✗	✗	✗	✓	n/a ⁵	✓	✓	✓
OIDC-centric solutions								
SPRESSO [62]	✗	◆ ²	✗	◆ ⁴	n/a ⁵	✓	✗	✗
PASTA [63]	✓	✗	◆ ³	◆ ⁴	n/a ⁵	✓	✗	✗
PESTO [64]	✓	✗	◆ ³	◆ ⁴	n/a ⁵	✓	✗	✗
UPPRESSO [9]	✗	✓	✓	✓	✗	✓	✗	✓
POIDC [65]	✗	✗	✗	◆ ⁴	n/a ⁵	✓	✗	✗
AIF [21]	✗	◆ ²	✗	◆ ⁴	n/a ⁵	✓	✓ ⁷	✗
OPPID [66]	✗	✓	◆ ³	✓	✗	✗	✓ ⁷	✗
MISO [20]	◆ ¹	✓	✗	◆ ⁴	n/a ⁵	✗	✓	✓
AC-based solutions								
PRIMA [10]	✗	✓	✓	◆ ⁴	✗	✗	✗	✗
EL PASSO [11]	✓	✓	✓	✓	✓	✓ ⁶	✗	✗
Lux et al. [12]	✓	✗	✓	✗	✗	✗	✗	✗
VC-SSO [13]	✓	✓	✓	✗	✗	✓ ⁶	✗	✗
U2SSO [15]	✓	✓	✓	✓	✗	✗	✗	✗
ARPSO [14]	✗	✓	✗	✓	✗	✗	✓	✓
VeriSSO (this work)	✓	✓	✓	✓	✓	✓ ⁶	✓	✓

◆¹: Resists IdP single-point failure, but requires trusted hardware at a central location. ◆²: Achieves RP hiding against IdP, but does not provide unlinkability against collusive RPs. ◆³: Achieves minimized attribute disclosure to RP, but remains transparent to IdP. ◆⁴: Achieves intra-RP linkability using a long-term user attribute instead of an RP-specific pseudonym. n/a⁵: Does not achieve anonymization or selective disclosure in the first place, thus not applicable for de-anonymization discussion. ✓⁶: Requires issuance of a new credential to each device; non-credential schemes use password and are device-agnostic. ✓⁷: Improves upon the Implicit Flow.

VIII. RELATED WORK

Previous works that provide user privacy and security enhancement in SSO can be generally categorized into *OIDC-centric* and *anonymous credential (AC)-based* schemes. Table II compares these schemes with VeriSSO with respect to a list of desired SSO properties.

A. OIDC-centric Solutions

The OIDC-centric approach aims to improve the original OIDC framework [1], upon either the Implicit Flow or ACF, for better user privacy protection and security properties. SPRESSO [62] makes one sign-in session indistinguishable from another by hiding RP's identity from the IdP through encrypting RP authenticators. PASTA [63] distributes token issuance across multiple servers using threshold oblivious pseudo-random functions (PRF) and secret-shared password hashes, resisting IdP single-point failure. PESTO [64] extends PASTA with proactive security, enabling token recovery. However, PASTA and PESTO are limited to Implicit Flow and lack user anonymization or RP hiding. UPPRESSO [9] is the first to achieve inter-RP unlinkability in the OIDC framework by issuing per-login, per-RP pseudonyms, so RPs cannot link users across different services.

POIDC [65], AIF [21], and OPPID [66] primarily focus on enhancing security and privacy in Implicit Flow. Specifically, POIDC [65] addresses these issues using cryptographic commitments on RP identifier. However, the commitment signature

in this approach does not prevent the IdP from linking the user's access to different RPs. AIF [21] leverages AC-based anonymous authentication for RPs (in contrast to AC-based user authentication). This approach, while fulfilling RP authentication, does not provide inter-RP unlinkability for users. OPPID [66] extends AIF to support inter-RP unlinkability by using pseudonyms computed by a variant of the HashDH PRF [67].

MISO [20] utilizes Trusted Execution Environment (TEE) to build a privacy-preserving intermediary, called a mixer, acting as a trusted IdP to the RP and an anonymous RP to the IdP, thus enabling anonymous authentication while preserving the ACF workflow. However, its reliance on TEE hardware that underlies the mixer introduces new risks of a single point of failure.

B. AC-based Solutions

AC-based solutions typically leverage the attribute-based authentication capabilities of anonymous credentials, enabling user privacy through selective disclosure. PRIMA [10] is an early work using cryptographic credentials instead of JWT tokens to enhance privacy, but it lacks RP authentication and requires continuous IdP availability. EL PASSO [11] is the first AC-based solution to achieve inter-RP unlinkability and lawful de-anonymization. It requires the IdP to blindly issue access credentials to users, who in turn use the credentials to selectively disclose attributes to RPs. EL PASSO also achieves intra-RP linkability to reduce the Sybil risks by generating the

same pseudonym for each user-RP pair, which incentivized our similar design. It, however, does not address the IdP single point of failure or provide RP authentication, and only works for Implicit Flow.

Lux et al. [12] and VC-SSO [13] leverage VCs to achieve decentralization of identity (no IdP single-point-of-failure) and selective disclosure under the self-sovereign identity (SSI) principles. VC-SSO further achieves inter-RP unlinkability through its ZKP component. While similar to our work in VC usage, these two solutions do not provide intra-RP linkability or lawful de-anonymization.

Recently, U2SSO [15] proposes to use anonymous self credentials (ASC) (an AC variant) for SSO that does not require an IdP for issuance/registration. Like EL PASSO, it is yet to support RP authentication and burdens the RP to manage the authentication service in-house. The above AC-based SSO schemes generally follow the Implicit Flow but do not support the ACF workflow. ARPSO [14] utilizes ACs for RP authentication, in a similar ideal to AIF [21], while also achieving inter-RP unlinkability for users and supporting ACF; it still requires IdP's real-time involvement, which poses a single-point failure risk.

Ongoing OpenID Foundation efforts [68], [69] aim to standardize VC-based SSO, transitioning from centralized IdPs toward user-controlled decentralized identity models. They propose to leverage VCs for user-centric identity management with decentralized control and enhanced privacy, which partly motivated our work.

IX. CONCLUSION

We present VeriSSO, an SSO protocol that leverages VC-based distributed authentication and threshold signatures to achieve user privacy and compatibility with the incumbent OIDC ACF workflow. VeriSSO allows users to control their information disclosure in SSO sessions while preserving user experience like traditional methods. Unlike other AC-based SSO schemes, VeriSSO crucially supports RP authentication and requires minimal architectural changes on the RP side, providing easier compatibility and migration from the existing OIDC workflow. We introduce a distributed authentication committee to take over the responsibilities of the centralized IdP. This design enables VeriSSO to handle single points of failure at IdP, malicious RP access, RP-IdP collusion, and identity token issuance. Our cloud-based experiments show that a complete VeriSSO authentication flow disclosing 50 user attributes can finish within 30 ms, demonstrating feasibility in practical use.

ACKNOWLEDGMENT

This work was supported in part by the US National Science Foundation (NSF) under grant numbers 2247561, 2433905, and 2442382. Ifteher Alom was also supported by the Light-house Beacon Foundation (LBF) Graduate Fellowship at the University of Kentucky.

REFERENCES

- [1] O. Foundation, "What are openid specifications." <https://openid.net/developers/specs/>, 2023. Accessed Online: 2023-08-17.
- [2] auth0, "What is oauth 2.0?." <https://auth0.com/intro-to-iam/what-is-oauth-2>, 2023. Accessed Online: 2023-08-17.
- [3] B. Krishnamurthy, D. Malandrino, and C. E. Wills, "Measuring privacy loss and the impact of privacy protection in web browsing," in *Proceedings of the 3rd Symposium on Usable Privacy and Security*, pp. 52–63, 2007.
- [4] E. Maler and D. Reed, "The venn of identity: Options and issues in federated identity management," *IEEE security & privacy*, vol. 6, no. 2, pp. 16–23, 2008.
- [5] S.-T. Sun, E. Pospisil, I. Muslukhov, N. Dindar, K. Hawkey, and K. Beznosov, "What makes users refuse web single sign-on? an empirical investigation of openid," in *Proceedings of the seventh symposium on usable privacy and security*, pp. 1–20, 2011.
- [6] S.-T. Sun, E. Pospisil, I. Muslukhov, N. Dindar, K. Hawkey, and K. Beznosov, "Investigating users' perspectives of web single sign-on: Conceptual gaps and acceptance model," *ACM Transactions on Internet Technology (TOIT)*, vol. 13, no. 1, pp. 1–35, 2013.
- [7] R. Gafni and D. Nissim, "To social login or not login? exploring factors affecting the decision," *Issues in Informing Science and Information Technology*, vol. 11, no. 1, pp. 57–72, 2014.
- [8] M. Uruña, A. Muñoz, and D. Larrabeiti, "Analysis of privacy vulnerabilities in single sign-on mechanisms for multimedia websites," *Multimedia Tools and Applications*, vol. 68, pp. 159–176, 2014.
- [9] C. Guo, J. Lin, Q. Cai, W. Wang, F. Li, Q. Wang, J. Jing, and B. Zhao, "Upresso: Untraceable and unlinkable privacy-preserving single sign-on services," *arXiv preprint arXiv:2110.10396*, 2021.
- [10] M. R. Asghar, M. Backes, and M. Simeonovski, "Prima: Privacy-preserving identity and access management at internet-scale," in *2018 IEEE International Conference on Communications (ICC)*, pp. 1–6, IEEE, 2018.
- [11] Z. Zhang, M. Król, A. Sonnino, L. Zhang, and E. Rivière, "El passo: Efficient and lightweight privacy-preserving single sign on," *Proceedings on Privacy Enhancing Technologies*, vol. 2021, no. 2, pp. 70–87, 2021.
- [12] Z. A. Lux, D. Thatmann, S. Zickau, and F. Beierle, "Distributed-ledger-based authentication with decentralized identifiers and verifiable credentials," in *2020 2nd Conference on Blockchain Research & Applications for Innovative Networks and Services (BRAINS)*, pp. 71–78, IEEE, 2020.
- [13] A. D. Johnson, I. Alom, and Y. Xiao, "Rethinking single sign-on: A reliable and privacy-preserving alternative with verifiable credentials," in *Proceedings of the 10th ACM Workshop on Moving Target Defense*, pp. 25–28, 2023.
- [14] J. He, L. Lei, Y. Wang, P. Wang, and J. Jing, "Arpsso: An oidc-compatible privacy-preserving sso scheme based on rp anonymization," in *European Symposium on Research in Computer Security*, pp. 268–288, Springer, 2024.
- [15] J. Alupotha, M. Barbaraci, I. Kaklamanis, A. Rawat, C. Cachin, and F. Zhang, "Anonymous self-credentials and their application to single-sign-on," *Cryptology ePrint Archive*, 2025.
- [16] J. Camenisch and A. Lysyanskaya, "An efficient system for non-transferable anonymous credentials with optional anonymity revocation," in *Advances in Cryptology—EUROCRYPT 2001: International Conference on the Theory and Application of Cryptographic Techniques Innsbruck, Austria, May 6–10, 2001 Proceedings 20*, pp. 93–118, Springer, 2001.
- [17] J. CAMENISCH, "Signature schemes and anonymous credentials from bilinear maps," in *Advances in Cryptology-CRYPTO 2004*, pp. 56–72, Springer-Verlag, 2004.
- [18] C. Garman, M. Green, and I. Miers, "Decentralized anonymous credentials," *Cryptology ePrint Archive*, 2013.
- [19] J. Camenisch, M. Drijvers, and A. Lehmann, "Anonymous attestation using the strong diffie hellman assumption revisited," in *Trust and Trustworthy Computing: 9th International Conference, TRUST 2016, Vienna, Austria, August 29-30, 2016, Proceedings 9*, pp. 1–20, Springer, 2016.
- [20] R. Xu, S. Yang, F. Zhang, and Z. Fang, "Miso: Legacy-compatible privacy-preserving single sign-on using trusted execution environments," in *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*, pp. 352–372, IEEE, 2023.
- [21] M. Kroschewski and A. Lehmann, "Save the implicit flow? enabling privacy-preserving rp authentication in openid connect," *Proceedings on Privacy Enhancing Technologies*, 2023.

- [22] W3C, “Verifiable credentials data model v2.0.” <https://www.w3.org/TR/vc-data-model-2.0/>, 2024. accessed 9/2/2024.
- [23] Auth0, “Proof key for code exchange (pkce) in web applications with spring security.” <https://auth0.com/blog/pkce-in-web-applications-with-spring-security/>, 2024.
- [24] Okta, “Implement authorization by grant type.” <https://developer.okta.com/docs/guides/implement-grant-type/clientcreds/main/>, 2024.
- [25] OAuth, “OAuth 2.0 implicit grant.” <https://oauth.net/2/grant-types/implicit/>, 2024.
- [26] Okta, “The best practice around implicit in oauth 2.0 is changing.” <https://developer.okta.com/blog/2019/05/01/is-the-oauth-implicit-flow-dead>, 2024.
- [27] J. Sedlmeir, R. Smethurst, A. Rieger, and G. Fridgen, “Digital identities and verifiable credentials,” *Business & Information Systems Engineering*, vol. 63, no. 5, pp. 603–613, 2021.
- [28] O. Goldreich, *Foundations of cryptography: volume 2, basic applications*, vol. 2. Cambridge university press, 2001.
- [29] J. Camenisch, M. Drijvers, P. Dzurenda, and J. Hajny, “Fast keyed-verification anonymous credentials on standard smart cards,” in *ICT Systems Security and Privacy Protection: 34th IFIP TC 11 International Conference, SEC 2019, Lisbon, Portugal, June 25-27, 2019, Proceedings 34*, pp. 286–298, Springer, 2019.
- [30] J. Camenisch and E. Van Herreweghen, “Design and implementation of the idemix anonymous credential system,” in *Proceedings of the 9th ACM Conference on Computer and Communications Security*, pp. 21–30, 2002.
- [31] J. Camenisch and A. Lysyanskaya, “Dynamic accumulators and application to efficient revocation of anonymous credentials,” in *Advances in Cryptology—CRYPTO 2002: 22nd Annual International Cryptology Conference Santa Barbara, California, USA, August 18–22, 2002 Proceedings 22*, pp. 61–76, Springer, 2002.
- [32] D. Boneh, X. Boyen, and H. Shacham, “Short group signatures,” in *Annual international cryptography conference*, pp. 41–55, Springer, 2004.
- [33] M. H. Au, W. Susilo, and Y. Mu, “Constant-size dynamic k-taa,” in *Security and Cryptography for Networks: 5th International Conference, SCN 2006, Maiori, Italy, September 6-8, 2006. Proceedings 5*, pp. 111–125, Springer, 2006.
- [34] Okta, “Auth0 documentation.” <https://auth0.com/docs>, 2024.
- [35] O. Homepage, “Bls12-381: New zk-snark elliptic curve construction.” <https://www.onelogin.com/>, 2024.
- [36] O. C. D. C. Registration, “Openid.” https://openid.net/specs/openid-connect-registration-1_0-errata2.html, 2024.
- [37] I. Alom, R. M. Eshita, A. I. Harun, M. S. Ferdous, M. K. B. Shuhan, M. J. M. Chowdhury, and M. S. Rahman, “Dynamic management of identity federations using blockchain,” in *2021 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pp. 1–9, IEEE, 2021.
- [38] I. E. T. F. (IETF), “The bbs signature scheme.” <https://identity.foundation/bbs-signature/draft-irtf-cfrg-bbs-signatures.html>, 2024.
- [39] A. Fiat and A. Shamir, “How to prove yourself: Practical solutions to identification and signature problems,” in *Conference on the theory and application of cryptographic techniques*, pp. 186–194, Springer, 1986.
- [40] Y. Seurin, “On the exact security of schnorr-type signatures in the random oracle model,” in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pp. 554–571, Springer, 2012.
- [41] T. Attema, S. Fehr, and M. Klooß, “Fiat-shamir transformation of multi-round interactive proofs,” in *Theory of Cryptography Conference*, pp. 113–142, Springer, 2022.
- [42] M. Chase and A. Lysyanskaya, “On signatures of knowledge,” in *Advances in Cryptology—CRYPTO 2006: 26th Annual International Cryptology Conference, Santa Barbara, California, USA, August 20-24, 2006. Proceedings 26*, pp. 78–96, Springer, 2006.
- [43] J. Doerner, Y. Kondi, E. Lee, A. Shelat, and L. Tyner, “Threshold bbs+ signatures for distributed anonymous credential issuance,” in *2023 IEEE Symposium on Security and Privacy (SP)*, pp. 773–789, IEEE, 2023.
- [44] C. Mazzocca, A. Acar, S. Uluagac, R. Montanari, P. Bellavista, and M. Conti, “A survey on decentralized identifiers and verifiable credentials,” *IEEE Communications Surveys & Tutorials*, 2025.
- [45] T. ElGamal, “A public key cryptosystem and a signature scheme based on discrete logarithms,” *IEEE transactions on information theory*, vol. 31, no. 4, pp. 469–472, 1985.
- [46] S. Tessaro and C. Zhu, “Revisiting bbs signatures,” in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pp. 691–721, Springer, 2023.
- [47] D. Boneh and X. Boyen, “Short signatures without random oracles and the sdh assumption in bilinear groups,” *Journal of cryptology*, vol. 21, no. 2, pp. 149–177, 2008.
- [48] V. Shoup and R. Gennaro, “Securing threshold cryptosystems against chosen ciphertext attack,” *Journal of Cryptology*, vol. 15, no. 2, pp. 75–96, 2002.
- [49] Y. Tsionis and M. Yung, “On the security of elgamal based encryption,” in *International Workshop on Public Key Cryptography*, pp. 117–134, Springer, 1998.
- [50] P.-A. Fouque and D. Pointcheval, “Threshold cryptosystems secure against chosen-ciphertext attacks,” in *International Conference on the Theory and Application of Cryptology and Information Security*, pp. 351–368, Springer, 2001.
- [51] C. Du, H. Yu, Y. Xiao, Y. T. Hou, A. D. Keromytis, and W. Lou, “{UCBlocker}: Unwanted call blocking using anonymous authentication,” in *32nd USENIX Security Symposium (USENIX Security 23)*, pp. 445–462, 2023.
- [52] B. Alzahrani, “An information-centric networking based registry for decentralized identifiers and verifiable credentials,” *IEEE Access*, vol. 8, pp. 137198–137208, 2020.
- [53] Microsoft, “What is fido2?” <https://www.microsoft.com/en-us/security/business/security-101/what-is-fido2>, 2024.
- [54] W3C, “Verifiable credentials data model v2.0.” <https://www.w3.org/TR/vc-data-model-2.0/>, 2023. Accessed Online: 2023-08-22.
- [55] A. Narayanan and V. Shmatikov, “Robust de-anonymization of large sparse datasets,” in *2008 IEEE Symposium on Security and Privacy (sp 2008)*, pp. 111–125, IEEE, 2008.
- [56] C. Mazzocca, A. Acar, S. Uluagac, and R. Montanari, “{EVOKE}: Efficient revocation of verifiable credentials in {IoT} networks,” in *33rd USENIX Security Symposium (USENIX Security 24)*, pp. 1279–1295, 2024.
- [57] S. Bowe, “Bls12-381: New zk-snark elliptic curve construction.” <https://electriccoin.co/blog/new-snark-curve/>, 2024.
- [58] L. Harchandani, “Bbs and bbs+ signatures and protocols for proof of knowledge of signature.” https://crates.io/crates/bbs_plus, 2024.
- [59] V. C. Bruno Blanchet, “Proverif: Cryptographic protocol verifier in the formal model.” <https://bblanche.github.io/proverif/>, 2025.
- [60] D. Dolev and A. Yao, “On the security of public key protocols,” *IEEE Transactions on information theory*, vol. 29, no. 2, pp. 198–208, 2003.
- [61] J. Zhang, L. Yang, W. Cao, and Q. Wang, “Formal analysis of 5g eap-tls authentication protocol using proverif,” *IEEE access*, vol. 8, pp. 23674–23688, 2020.
- [62] D. Fett, R. Küsters, and G. Schmitz, “Spresso: A secure, privacy-respecting single sign-on system for the web,” in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, pp. 1358–1369, 2015.
- [63] S. Agrawal, P. Miao, P. Mohassel, and P. Mukherjee, “Pasta: password-based threshold authentication,” in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pp. 2042–2059, 2018.
- [64] C. Baum, T. Frederiksen, J. Hesse, A. Lehmann, and A. Yanai, “Pesto: proactively secure distributed single sign-on, or how to trust a hacked server,” in *2020 IEEE European Symposium on Security and Privacy (EuroS&P)*, pp. 587–606, IEEE, 2020.
- [65] S. Hammann, R. Sasse, and D. Basin, “Privacy-preserving openid connect,” in *Proceedings of the 15th ACM Asia conference on computer and communications security*, pp. 277–289, 2020.
- [66] M. Kroschewski, A. Lehmann, and C. Özbay, “Oppid: Single sign-on with oblivious pairwise pseudonyms,” *Cryptology ePrint Archive*, 2024.
- [67] S. Jarecki, A. Kiayias, and H. Krawczyk, “Round-optimal password-protected secret sharing and t-pake in the password-only model,” in *Advances in Cryptology—ASIACRYPT 2014: 20th International Conference on the Theory and Application of Cryptology and Information Security, Kaoshiung, Taiwan, ROC, December 7-11, 2014, Proceedings, Part II 20*, pp. 233–253, Springer, 2014.
- [68] O. Foundation, “What is openid for verifiable credentials.” <https://openid.net/sg/openid4vc/specifications/>, 2024.
- [69] K. N. Chadwick and J. Vercammen, “Openid for verifiable credentials,” *OpenID Foundation*, 2022.
- [70] P. S. Barreto and M. Naehrig, “Pairing-friendly elliptic curves of prime order,” in *International workshop on selected areas in cryptography*, pp. 319–331, Springer, 2005.
- [71] D. Chaum, “Blind signatures for untraceable payments,” in *Advances in Cryptology: Proceedings of Crypto 82*, pp. 199–203, Springer, 1983.
- [72] D. Schröder and D. Unruh, “Security of blind signatures revisited,” *Journal of Cryptology*, vol. 30, pp. 470–494, 2017.

- [73] J. Katz, J. Loss, and M. Rosenberg, “Boosting the security of blind signature schemes,” in *Advances in Cryptology–ASIACRYPT 2021: 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6–10, 2021, Proceedings, Part IV* 27, pp. 468–492, Springer, 2021.
- [74] S. Faust, C. Hazay, D. Kretzler, L. Rometsch, and B. Schlosser, “Non-interactive threshold bbs+ from pseudorandom correlations,” *Cryptology ePrint Archive*, 2023.
- [75] I. Foundation. <https://identity.foundation/bbs-signature/draft-blind-bbs-signatures.txt>, 2024.

APPENDIX

A. Preliminaries

Bilinear Maps. Let $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ be groups of prime order p . A map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ must satisfy the following properties:

- **Bilinearity:** $e_b(g_1^x, g_2^y) = e_b(g_1, g_2)^{xy}$ for all $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$, and $x, y \in \mathbb{Z}_p$.
- **Non-degeneracy:** For all generators $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$, $e_b(g_1, g_2) \neq 1$, generates $\mathbb{G}_T$.
- **Efficiency:** There exists an efficient algorithm to compute $e_b(a, b)$ for any $a \in \mathbb{G}_1$, $b \in \mathbb{G}_2$.

Type-3 Pairing. They allow for the most efficient and secure operations in $\mathbb{G}_1$ using BN curves [70]. We describe our scheme in a type-3 setting, i.e., assuming $\mathbb{G}_1 \neq \mathbb{G}_2$ and no efficiently computable homomorphism $\mathbb{G}_2 \rightarrow \mathbb{G}_1$ exists.

B. BBS Signature

In a typical setup, BBS signature builds on pairing-based cryptography (refer to Appendix A). We describe a non-interactive version of the BBS signature protocol from [38], [46] that is adopted by our system. Let $\mathbb{G}_1$ and $\mathbb{G}_2$ be bilinear groups of prime order p , g_1 and g_2 be generators of $\mathbb{G}_1$ and $\mathbb{G}_2$ respectively.

BBS.KeyGen :

- Randomly sample a vector of group elements $(h_1, \dots, h_L) \leftarrow \mathbb{G}_1^L$,
- and uniformly choose a **Secret Key** key $x \leftarrow \mathbb{Z}_p^*$.
- Then compute $w = g_2^x$, where g_2 is a generator of the group $\mathbb{G}_2$, and set **Public Key** as $(w, h_1, \dots, h_L) \in \mathbb{G}_2 \times \mathbb{G}_1^L$.

BBS.Sign :

- On the input of a VC attributes set $\vec{m} = (m_1, m_2, \dots, m_L) \in \mathbb{Z}_p^L$, and the Secret key $x \in \mathbb{Z}_p^*$.
- Select a random nonce, $e \leftarrow \mathbb{Z}_p$.
- and compute $B = g_1 \prod_{i=1}^L h_i^{m_i}$ and $A = B^{\frac{1}{e+x}}$.
- The **signature** σ is the tuple $(A, e) \in \mathbb{G}_1 \times \mathbb{Z}_p$.

BBS.VerifySign :

- On the input of Public key $(w, h_1, \dots, h_L) \in \mathbb{G}_2 \times \mathbb{G}_1^L$, attributes set $\vec{m} = (m_1, m_2, \dots, m_L) \in \mathbb{Z}_p^L$, and signature $(A, e) \in \mathbb{G}_1 \times \mathbb{Z}_p$.
- Compute the following pairings and check the bilinear equality $e_b(A, w g_2^e) \stackrel{?}{=} e_b(B, g_2)$.
- Accept the signature if the equality holds, reject otherwise.

C. Realizing $\mathcal{F}_{\text{BIVC}}$

A blind signature is a cryptographic scheme that allows the signer to sign a message without knowing its contents. Like most standard digital signatures, a blind signature is unforgeable and can be verified against the signer’s public key. *Strongly-blind signatures* are protocols in which the signer does not learn any useful information about the message or the signature, whereas in a *weakly-blind signature* it gets to know the signature. Most of the digital signature schemes based on RSA, ECDSA, BBS, etc., can be adapted to support blind signing by obscuring the message using a random factor commonly known as *blinding factor* [71], [72], [73]. The weakly blind signing properties of the BBS signature scheme, as in previous practices [46], [43], [74], [75] to facilitate blind VC creation where the signature is issued to a commitment of the messages/attributes.

In VeriSSO’s user registration phase, the IdP uses the BBS signature scheme as described in Appendix B for signing VCs. In VeriSSO, the user requests for signature on a set that includes a user-chosen secret u as a hidden attribute. We thus refer to the corresponding signature in our scheme as a *weak partially-blind signature*. In other words, the IdP issues a standard BBS signature on the combined set $\vec{m}$ consisting of user attributes, blinded user secret $\tilde{u}$, and IdP-assigned identifier γ . As the hidden secret u is part of the VC attributes, and the user knows the chosen u and the blinding exponent d , the unbinding operation does not require any complex mathematical operations. Also, during the presentation phase, u and γ are simply treated along with other undisclosed attributes and this preserves all the VeriSSO privacy requirements. However, our modular design will allow integration of strongly-blind VC issuance in which the $\mathcal{F}_{\text{BIVC}}$.unblind functionality may be differently implemented.

D. Realizing $\mathcal{F}_{\text{SDVP}}$

VeriSSO uses the BBS signature scheme [38] and the SPoK primitive to construct the VC and VP mechanism, which enables the authentication phase to prove the correctness of selective disclosure of credential attributes, as well as the correctness of the ElGamal encryption of the user identifier.

1) *ZKP of BBS signature:* BBS signature allows a user/prover to prove validity of a signature without actually revealing it or fully disclosing the signed attributes. Given that the prover has signature $\sigma = (A, e) \in \mathbb{G}_1 \times \mathbb{Z}_p$ and can compute $B = g_1 \prod_{i=1}^L h_i^{m_i}$ for a vector $\vec{m}$ of L messages. It then executes the following routines:

BBS.SPoK :

- To selectively disclose attributes $att_D = \{m_i\}_{i \in D}$ and hide $att_H = \{m_i\}_{i \notin D}$, select a random nonce $r \leftarrow \mathbb{Z}_p^*$ and:
- Randomize the original signature σ as $A' = A^r$, $B' = B^r$, $\bar{A} = (A^{-e} \cdot B)^r = (A')^{-e} \cdot B'$.
- The prover then computes:

$$\pi_1 = \frac{\bar{A}}{B'} \quad (4)$$

$$\pi_2 = g_1 \prod_{i \in D} h_i^{m_i} \quad (5)$$

- The prover then sends $(att_D, A', \bar{A}, B', \pi_1, \pi_2)$ to the verifier.

BBS.VerifySPoK :

- The verifier receives $(att_D, A', \bar{A}, B', \pi_1, \pi_2)$ and verifies the bilinear equality: $e_b(A', w) \stackrel{?}{=} e_b(\bar{A}, g_2)$ and the proof Π given by:

$$\text{SPoK} \left\{ (att_H, A, e, r) : \frac{\bar{A}}{B'} = \pi_1 \wedge g_1 \prod_{i \in D} h_i^{m_i} = \pi_2 \right\} \quad (6)$$

- Accept if the bilinear equality and SPoK hold.

2) *RP-Specific Pseudonym Generation:* On the input of user secret u and RP domain $domain_{RP}$ (from $auth_{RP}$), pseudonym $\tilde{u}$ is produced as:

$$\tilde{u} = \zeta^u \quad (7)$$

where $\zeta = H(domain_{RP})$ and H is a cryptographically secure hash function.

Then the user computes a ZKP of his knowledge and possession of the original identifiers γ and u as:

$$\phi = \text{ZKP}\{E(\gamma) = (g^\alpha, y^\alpha \gamma) \wedge \tilde{u} = \zeta^u\} \quad (8)$$

where $E(\gamma) = (g^\alpha, y^\alpha \gamma)$ is an ElGamal encryption of the permanent identifier γ . Elements g is a generator of group $\mathbb{G}_1$, α is a random scalar, and y is the aggregated key of a threshold number of decryption authorities (which are chosen from the ASs in our system with additional privileges for accountability cases).

E. Threshold BBS+ Signature (Realizing $\mathcal{F}_{\text{TIDT}}^{(t,N)}$)

We provide an overview of the threshold BBS+ scheme from [43], which is used for realizing $\mathcal{F}_{\text{TIDT}}^{(t,N)}$ that generates the identity token τ_{ac} issuance phase of VeriSSO.

In the setup phase, the MPC servers collectively generate the shares of a secret key x_i and commit to a set of $(L+1)$ random scalars. The signing phase is initiated by a client who submits a signing request for an L -message vector to a certain number of MPC servers. At first, the servers select a secret blinding factor r_i , commit to random scalar e , and compute functions involving (r_i, x_i) . The client combines results from t threshold number of servers in the aggregation phase to produce a valid threshold BBS+ signature given by $\sigma_t = (\tilde{A}, e, s)$, where,

$$\tilde{A} = \frac{\sum_{i \in N} R_i}{\sum_{i \in N} U_i} \quad (9)$$

Specifically, in the case of VeriSSO, each AS of the signing parties, returns R_i and U_i , where

$$R_i = r_i \cdot (g_1 + s \cdot h_1 + \sum_{k \in [L+1]} m_k \cdot h_k) \quad (10)$$

$$U_i = r_i \cdot (x + e)$$

And x is the secret key, e and s are random scalars, r_i is a secret share of a random scalar r , and L is the number of messages to be signed in the attribute set $(m_1, m_2, \dots, m_L)$.

F. Security Assumptions

The q -SDH assumption, as introduced by Boneh and Boyen for type-3 pairing settings [47], can be stated as follows [19]:

Definition 5 (q -Strong Diffie-Hellman Assumption (q -SDH) [19]). *Given the tuple*

$$(g_1, g_1^x, g_1^{x^2}, \dots, g_1^{x^q}, g_2, g_2^x) \in \mathbb{G}_1^{q+1} \times \mathbb{G}_2^2,$$

where $\mathbb{G}_1$ and $\mathbb{G}_2$ are cyclic groups of prime order p with generators g_1 and g_2 , and $x \in \mathbb{Z}_p$ is randomly chosen and unknown, it is computationally infeasible for any probabilistic polynomial-time (PPT) adversary to produce a pair

$$(c, g_1^{1/(x+c)}) \in \mathbb{Z}_p \setminus \{-x\} \times \mathbb{G}_1$$

Here, $c \neq -x$ ensures that $1/(x+c)$ is well-defined.

G. Unlinkability Analysis

$$\mathcal{G}^{RPPF}(1^\lambda, \mathcal{B})$$

1. $domain_{RP} \leftarrow auth_{RP}$
2. $b \xleftarrow{\$} \{0, 1\}$
3. **if** $b = 0$, $f \leftarrow RPPF(domain_{RP}, \gamma)$
4. **if** $b = 1$, $f \leftarrow rand()$
5. $x_1, x_2, \dots, x_i \leftarrow \mathcal{B}$
6. $y_i = f(x_i); i = 1, 2, \dots$
7. $b' \xleftarrow{\$} \mathcal{B}(y_1, y_2, \dots, y_i)$.
8. **return** ‘success’ if $b = b'$; ‘fail’ otherwise.

Fig. 11: RPPF Security Game.

$$\mathcal{G}^{RPPF}(1^\lambda, \mathcal{B}, \mathcal{A})$$

1. $domain_{RP} \leftarrow auth_{RP}$
2. $b \xleftarrow{\$} \{0, 1\}$.
3. **if** $b = 0$, $f \leftarrow RPPF(domain_{RP}, \gamma)$
4. **if** $b = 1$, $f \leftarrow rand()$
5. $domain_{RP_m}, \gamma_n \leftarrow \mathcal{A}; m = 1, 2, \dots; n = 1, 2, \dots$
6. $x_{m,n} := RPPF(domain_{RP_m}, \gamma_n)$
7. $x_{1,1}, x_{1,2}, \dots, x_{m,n} \leftarrow \mathcal{B}$
8. $y_{m,n} = f(x_{m,n}); m = 1, 2, \dots; n = 1, 2, \dots$
9. $u_{m,n} := y_{m,n}$
10. $b' \xleftarrow{\$} \mathcal{A}(u_{1,1}, u_{1,2}, \dots, u_{m,n})$.
11. **return** ‘success’ if $b = b'$; ‘fail’ otherwise.

Fig. 12: Reduction from the RPPF Game.